

# Developer Specification (DEV\_SPEC)

版本：0.1 — 文档结构草案

## 目录

- 项目概述
- 核心特点
- 技术选型
- 测试方案
- 系统架构与模块设计
- 项目排期
- 可扩展性与未来展望

## 1. 项目概述

本项目基于多阶段检索增强生成 (RAG, Retrieval-Augmented Generation) 与模型组合平台 (MCP, Model Composition Platform) 设计，目标是搭建一个可扩展、高可观测、易迭代的智能问答与知识检索框架。

### 设计理念 (Design Philosophy)

#### 核心定位：自学与教学同步 (Learning by Teaching)

本项目是我个人技术学习、丰富简历、备战面试的实战历程，同时也是一份同步教学的开源资源。我相信“教是最好的学”——在整理代码、撰写文档、录制视频的过程中，我自己对 RAG 的理解也在不断深化。希望这份“边学边教”的成果能够帮助到更多同样在求职路上的朋友。

本项目不仅是一个功能完备的智能问答框架，更是一个专为 **RAG 技术学习与面试求职** 设计的实战平台：

#### ① 实战驱动学习 (Learn by Doing)

项目架构本身就是 RAG 面试题的“活体答案”。我们将经典面试考点直接融入代码设计，通过动手实践来巩固理论知识：

- 分层检索 (Hierarchical Retrieval)
- Hybrid Search (BM25 + Dense Embedding)
- Rerank 重排序机制
- Embedding 策略与优化
- RAG 性能评测 (Ragas/DeepEval)

#### ② 开箱即用与深度扩展并重 (Plug-and-Play & Extensible)

- 开箱即用**：提供 MCP 标准接口，可直接对接 Copilot/Claude，拿到项目即可运行体验。
- 深度扩展**：保留完全模块化的内部结构，方便开发者替换组件、魔改算法，作为具备深度的个人简历项目。
- 扩展指引**：文档中会明确指出各模块的扩展方向与建议，帮助你在掌握基础后继续深入迭代。

3 配套教学资源 (Comprehensive Learning Materials)

我会提供三位一体的配套学习资源，帮助你快速吃透项目：

| 资源类型                                                                                   | 内容说明                           |
|----------------------------------------------------------------------------------------|--------------------------------|
|  技术文档 | 架构设计文档、技术选型说明、模块详解             |
|  代码示范 | 带详细注释的源码、关键模块的 Step-by-step 实现 |
|  视频讲解 | RAG 核心知识点回顾、代码细节精讲、环境配置教程      |

4 学习路线与面试指南 (Study Guide & Interview Prep)

针对每个模块，我会整理：

-  **知识点清单**：这块涉及哪些理论知识需要提前学习（如 BM25 原理、FAISS 索引类型、Cross-Encoder vs Bi-Encoder）
-  **高频面试题**：结合项目代码讲解常见面试问题及参考答案
-  **简历撰写建议**：如何将本项目的亮点写进简历，突出技术深度

5 社区交流与持续迭代 (Community & Iteration)

- 经验分享**：我自己的面试经历、大家使用本项目面试的反馈，都会汇总沉淀
- 问题讨论**：一起探讨"如何将本项目写进简历"、"针对本项目的面试题怎么答"
- 持续更新**：从代码 → 入股知识 → 面试技巧，形成完整的求职知识库，帮助大家更好地拿到 Offer 

2. 核心特点

RAG 策略与设计亮点

本项目在 RAG 链路的关键环节采用了经典的工程化优化策略，平衡了检索的查准率与查全率，具体思想如下：

- 分块策略 (Chunking Strategy)**：采用智能分块与上下文增强，为高质量检索打下基础。
  - 智能分块**：摒弃机械的定长切分，采用语义感知的切分策略以保留完整语义；
  - 上下文增强**：为 Chunk 注入文档元数据（标题、页码）和图片描述（Image Caption），确保检索时不仅匹配文本，还能感知上下文。
- 粗排召回 (Coarse Recall / Hybrid Search)**：采用 **混合检索** 策略作为第一阶段召回，快速筛选候选集。
  - 结合 **稀疏检索 (Sparse Retrieval/BM25)** 利用关键词精确匹配，解决专有名词查找问题；
  - 结合 **稠密检索 (Dense Retrieval/Embedding)** 利用语义向量，解决同义词与模糊表达问题；
  - 两者互补，通过 RRF (Reciprocal Rank Fusion) 算法融合，确保查全率与查准率的平衡。
- 精排重排 (Rerank / Fine Ranking)**：在粗排召回的基础上进行深度语义排序。
  - 采用 Cross-Encoder（专用重排模型）或 LLM Rerank（可选后端）对候选集进行逐一打分，识别细微的语义差异。
  - 通过 **"粗排(低成本泛召回) -> 精排(高成本精过滤)"** 的两段式架构，在不牺牲整体响应速度的前提下大幅提升 Top-Results 的精准度。

全链路可插拔架构 (Pluggable Architecture)

鉴于 AI 技术的快速演进，本项目在架构设计上追求**极致的灵活性**，拒绝与特定模型或供应商强绑定。**整个系统**（不仅是 RAG 链路）的每一个核心环节均定义了抽象接口，支持“乐高积木式”的自由替换与组合：

- **LLM 调用层插拔 (LLM Provider Agnostic)：**
  - 核心推理 LLM 通过统一的抽象接口封装，支持**多协议**无缝切换：
    - **Azure OpenAI**：企业级 Azure 云端服务，符合合规与安全要求；
    - **OpenAI API**：直接对接 OpenAI 官方接口；
    - **本地模型**：支持 Ollama、vLLM、LM Studio 等本地私有化部署方案；
    - **其他云服务**：DeepSeek、Anthropic Claude 等第三方 API。
  - 通过配置文件一键切换后端，**零代码修改**即可完成 LLM 迁移，便于成本优化、隐私合规或 A/B 测试。
- **Embedding & Rerank 模型插拔 (Model Agnostic)：**
  - Embedding 模型与 Rerank 模型同样采用统一接口封装；
  - 支持云端服务（OpenAI Embedding, Cohere Rerank）与本地模型（Sentence-Transformers, BGE）自由切换。
- **RAG Pipeline 组件插拔：**
  - **Loader（解析器）**：支持 PDF、Markdown、Code 等多种文档解析器独立替换；
  - **Smart Splitter（切分策略）**：语义切分、定长切分、递归切分等策略可配置；
  - **Transformation（元数据/图文增强逻辑）**：OCR、Image Captioning 等增强模块可独立配置。
- **检索策略插拔 (Retrieval Strategy)：**
  - 支持动态配置纯向量、纯关键词或混合检索模式；
  - 支持灵活更换向量数据库后端（如从 Chroma 迁移至 Qdrant、Milvus）。
- **评估体系插拔 (Evaluation Framework)：**
  - 评估模块不锁定单一指标，支持挂载不同的 Evaluator（如 Ragas, DeepEval）以适应不同的业务考核维度。

这种设计确保开发者可以**零代码修改**即可进行 A/B 测试、成本优化或隐私迁移，使系统具备极强的生命力与环境适应性。

## MCP 生态集成 (Copilot / ReSearch)

本项目的核心设计完全遵循 Model Context Protocol (MCP) 标准，这使得它不仅是一个独立的问答服务，更是一个即插即用的知识上下文提供者。

- **工作原理：**
  - 我们的 Server 作为一个 **MCP Server** 运行，暴露一组标准的 **tools** 和 **resources** 接口。
  - **MCP Clients**（如 GitHub Copilot, ReSearch Agent, Claude Desktop 等）可以直接连接到这个 Server。
  - **无缝接入**：当你在 GitHub Copilot 中提问时，Copilot 作为一个 MCP Host，能够自动发现并调用我们的 Server 提供的工具（如 **search\_documentation**），获取我们内置的私有文档知识，然后结合这些上下文来回答你的问题。
- **优势：**

- **零前端开发**：无需为知识库开发专门的 Chat UI，直接复用开发者已有的编辑器（VS Code）和 AI 助手。
- **上下文互通**：Copilot 可以同时看到你的代码文件和我们的知识库内容，进行更深度的推理。
- **标准兼容**：任何支持 MCP 的 AI Agent（不仅是 Copilot）都可以即刻接入我们的知识库，一次开发，处处可用。

## 多模态图像处理 (Multimodal Image Processing)

本项目采用了经典的 **"Image-to-Text" (图转文)** 策略来处理文档中的图像内容，实现了低成本且高效的多模态检索：

- **图像描述生成 (Captioning)**：利用 LLM 的视觉能力，自动提取文档中插图的核心信息，并生成详细的文字描述（Caption）。
- **统一向量空间**：将生成的图像描述文字直接嵌入到文档文本块（Chunk）中进行向量化。
- **优势**：
  - **架构统一**：无需引入复杂的 CLIP 等多模态向量库，复用现有的纯文本 RAG 检索链路即可实现“搜文字出图”。
  - **语义对齐**：通过 LLM 将图像的视觉特征转化为语义理解，使用户能通过自然语言精准检索到图表、流程图等视觉信息。

## 可观测性与评估体系 (Observability & Evaluation)

针对 RAG 系统常见的“黑盒”问题，本项目致力于让每一次生成过程都**透明可见且可量化**：

- **全链路白盒化 (White-box Tracing)**：
  - 记录并可视化 RAG 流水线的每一个中间状态：从 **Query** 改写，到 **Hybrid Search** 的初步召回列表，再到 **Reranker** 的打分排序，最后到 **LLM** 的 Prompt 构建。
  - 开发者可以清晰看到“系统为什么选了这个文档”以及“Rerank 起了什么作用”，从而精准定位坏 Case。
- **自动化评估闭环 (Automated Evaluation)**：
  - 集成 Ragas 等评估框架，为每一次检索和生成计算“体检报告”（如召回率 Hit Rate、准确性 Faithfulness 等指标）。
  - 拒绝“凭感觉”调优，建立基于数据的迭代反馈回路，确保每一次策略调整（如修改 Chunk Size 或更换 Reranker）都有量化的分数支撑。

## 业务可扩展性 (Extensibility for Your Own Projects)

本项目采用**通用化架构设计**，不仅是一个开箱即用的知识问答系统，更是一个可以快速适配各类业务场景的**扩展基座**：

- **Agent 客户端扩展 (Build Your Own Agent Client)**：
  - 本项目的 MCP Server 天然支持被各类 Agent 调用，你可以基于此构建属于自己的 Agent 客户端：
    - **学习 Agent 开发**：通过实现一个调用本 Server 的 Agent，深入理解 Agent 的核心概念（Tool Calling、Chain of Thought、ReAct 模式等）；
    - **定制业务 Agent**：结合你的具体业务需求，开发专属的智能助手（如代码审查 Agent、文档写作 Agent、客服问答 Agent）；

- **多 Agent 协作**：将本 Server 作为知识检索 Agent，与其他功能 Agent（如代码生成、任务规划）组合，构建复杂的 Multi-Agent 系统。
- **业务场景快速适配 (Adapt to Your Domain)**：
  - **数据层扩展**：只需替换数据源（接入你自己的文档、数据库、API），即可将本系统改造为你的私有知识库；
  - **检索逻辑定制**：基于可插拔架构，轻松调整检索策略以适配不同业务特点（如电商搜索偏重关键词、法律文档偏重语义）；
  - **Prompt 模板定制**：修改系统 Prompt 和输出格式，使其符合你的业务风格与专业术语。
- **学习与实战并重 (Learn While Building)**：
  - 通过扩展本项目，你将同步掌握：
    - **Agent 架构设计**：Function Calling、Tool Use、Memory 管理等核心概念；
    - **LLM 应用工程化**：Prompt Engineering、Token 优化、流式输出等实战技能；
    - **系统集成能力**：如何将 AI 能力嵌入现有业务系统，构建端到端的智能应用。

这种设计让本项目不仅是“学完即弃”的 Demo，而是可以**持续迭代、真正落地**的工程化模板，帮助你学到的知识转化为实际项目经验。

## 3. 技术选型

### 3.1 RAG 核心流水线设计

#### 3.1.1 数据摄取流水线

**目标**：使用 LlamaIndex 的 Ingestion Pipeline 构建统一、可配置且可观测的数据导入与分块（chunking）能力，覆盖文档加载、格式解析、语义切分、多模态增强、嵌入计算、去重与批量上载到向量存储。该能力应是可重用的库模块，便于在 `ingest.py`、离线批处理和测试中调用。

- **为什么选 LlamaIndex**：
  - 提供成熟的 Ingestion / Node parsing 抽象，易于插入自定义 Transform（例如 ImageCaptioning）。
  - 与主流 embedding provider 有良好适配器生态，架构中统一使用 Chroma 作为向量存储。
  - 支持可组合的 Loader -> Splitter -> Transform -> Embed -> Upsert 流程，便于实现可观测的流水线。

设计要点：

- **明确分层职责**：
  - **Loader**：负责把原始文件解析为统一的 **Document** 对象（`text + metadata`）。在当前阶段，仅实现 **PDF 格式的 Loader**。
    - 统一输出格式采用规范化 Markdown 作为 `Document.text`：这样可以更好的配合后面的 Splitter（Langchain RecursiveCharacterTextSplitter）方法产出高质量切块。
    - Loader 同时抽取/补齐基础 metadata（如 `source_path`, `doc_type=pdf`, `page`, `title/heading_outline`, `images` 引用列表等），为定位、回溯与后续 Transform 提供依据。
  - **Splitter**：基于 Markdown 结构（标题/段落/代码块等）与参数配置把 **Document** 切为若干 Chunk，保留原始位置与上下文引用。

- Transform：可插入的处理步骤（ImageCaptioning、OCR、code-block normalization、html-to-text cleanup 等）。Transform 可以选择把额外信息追加到 chunk.text 或放入 chunk.metadata（推荐默认追加到 text 以保证检索覆盖）。
- Embed & Upsert：按批次计算 embedding，并上载到向量存储；支持向量 + metadata 上载，并提供幂等 upsert 策略（基于 id/hash）。
- Dedup & Normalize：在上载前运行向量/文本去重与哈希过滤，避免重复索引。

关键实现要素：

- Loader（统一格式与元数据）
  - 前置去重 (Early Exit / File Integrity Check)：
    - 机制：在解析文件前，计算原始文件的 SHA256 哈希指纹。
    - 动作：检索 `ingestion_history` 表，若发现相同 Hash 且状态为 `success` 的记录，则认定该文件未发生变更，直接跳过后续所有处理（解析、切分、LLM重写），实现**零成本 (Zero-Cost)** 的增量更新。
  - 解析与标准化：
    - 当前范围：仅实现 **PDF -> canonical Markdown 子集** 的转换。
  - 技术选型（Python PDF -> Markdown）：
    - 首选：**MarkItDown**（作为默认 PDF 解析/转换引擎）。优点是直接产出 Markdown 形态文本，便于与后续 `RecursiveCharacterTextSplitter` 的 `separators` 配合。
  - 输出标准 Document：`id|source|text(markdown)|metadata`。metadata 至少包含 `source_path, doc_type, title/heading_outline, page/slide`（如适用），`images`（图片引用列表）。
  - Loader 不负责切分：只做“格式统一 + 结构抽取 + 引用收集”，确保切分策略可独立迭代与度量。
- Splitter（LangChain 负责切分；独立、可控）
  - 实现方案：使用 LangChain 的 `RecursiveCharacterTextSplitter` 进行切分。
    - 优势：该方法对 Markdown 文档的结构（标题、段落、列表、代码块）有天然的适配性，能够通过配置语义断点（Separators）实现高质量、语义完整的切块。
  - Splitter 输入：Loader 产出的 Markdown Document。
  - Splitter 输出：若干 `Chunk`（或 Document-like chunks），每个 chunk 必须携带稳定的定位信息与来源信息：`source, chunk_index, start_offset/end_offset`（或等价定位字段）。
- Transform & Enrichment（结构转换与深度增强）本阶段是 ETL 管道的核心“智力”环节，负责将 Splitter 产出的非结构化文本块转化为结构化、富语义的智能切片（Smart Chunk）。
  - 结构转换 (Structure Transformation)：将原始的 `String` 类型数据转化为强类型的 `Record/Object`，为下游检索提供字段级支持。
  - 核心增强策略：
    1. 智能重组 (Smart Chunking & Refinement)：
      - 策略：利用 LLM 的语义理解能力，对上一阶段“粗切分”的片段进行二次加工。
      - 动作：合并逻辑上紧密相关但被物理切断的段落，剔除无意义的页眉页脚或乱码（去噪），确保每个 Chunk 是自包含 (Self-contained) 的语义单元。
    2. 语义元数据注入 (Semantic Metadata Enrichment)：
      - 策略：在基础元数据（路径、页码）之上，利用 LLM 提取高维语义特征。
      - 产出：为每个 Chunk 自动生成 `Title`（精准小标题）、`Summary`（内容摘要）和 `Tags`（主题标签），并将其注入到 Metadata 字段中，支持后续的混合检索与精确过



滤。

### 3. 多模态增强 (Multimodal Enrichment / Image Captioning) :

- 策略：扫描文档片段中的图像引用，调用 Vision LLM ( 如 GPT-4o ) 进行视觉理解。
  - 动作：生成高保真的文本描述 ( Caption ) ，描述图表逻辑或提取截图文字。
  - 存储：将 Caption 文本“缝合”进 Chunk 的正文或 Metadata 中，打通模态隔阂，实现“搜文出图”。
- 工程特性：Transform 步骤设计为原子化与幂等操作，支持针对特定 Chunk 的独立重试与增量更新，避免因 LLM 调用失败导致整个文档处理中断。

#### • Embedding (双路向量化)

- 增量计算 (Incremental Embedding / Cost Optimization) :
  - 策略：在调用昂贵的 Embedding API 之前，计算 Chunk 的内容哈希 ( Content Hash ) 。仅针对数据库中不存在的新内容哈希执行向量化计算，对于文件名变更但内容未变的片段，直接复用已有向量，显著降低 API 调用成本。
- 核心策略：为了支持高精度的混合检索 ( Hybrid Search ) ，系统对每个 Chunk 并行执行双路编码计算。
  - **Dense Embeddings ( 语义向量 )**：调用 Embedding 模型 ( 如 OpenAI text-embedding-3 或 BGE ) 生成高维浮点向量，捕捉文本的深层语义关联，解决“词不同意同”的检索难题。
  - **Sparse Embeddings ( 稀疏向量 )**：利用 BM25 编码器或 SPLADE 模型生成稀疏向量 ( Keyword Weights ) ，捕捉精确的关键词匹配信息，解决专有名词查找问题。
- 批处理优化：所有计算均采用 `batch_size` 驱动的批处理模式，最大化 CPU 利用率并减少网络 RTT。

#### • Upsert & Storage (索引存储)

- 存储后端：统一使用向量数据库 ( 如 Chroma/Qdrant ) 作为存储引擎，同时持久化存储 Dense Vector、Sparse Vector 以及 Transform 阶段生成的富 Metadata。
- **All-in-One 存储策略**：执行原子化存储，每条记录同时包含：
  1. **Index Data**: 用于计算相似度的 Dense Vector 和 Sparse Vector。
  2. **Payload Data**: 完整的 Chunk 原始文本 (Content) 及 Metadata。 **机制优势**：确保检索命中 ID 后能立即取回对应的正文内容，无需额外的查库操作 (Lookup)，保障了 Retrieve 阶段的毫秒级响应。
- 幂等性设计 (Idempotency) :
  - 为每个 Chunk 生成全局唯一的 `chunk_id`，生成算法采用确定的哈希组合：  
`hash(source_path + section_path + content_hash)`。
  - 写入时采用 "Upsert" ( 更新或插入 ) 语义，确保同一文档即使被多次处理，数据库中也永远只有一份最新副本，彻底避免重复索引问题。
- 原子性保证：以 Batch 为单位进行事务性写入，确保索引状态的一致性。

### 3.1.2 检索流水线 (Retrieval Pipeline)

本模块实现核心的 RAG 检索引擎，采用“多阶段过滤 (Multi-stage Filtering)”架构，负责接收已消歧的独立查询 ( Standalone Query ) ，并精准召回 Top-K 最相关片段。

#### • Query Processing (查询预处理)

- 核心假设：输入 Query 已由上游 ( Client/MCP Host ) 完成会话上下文补全 ( De-referencing ) ，不仅如此，还进行了指代消歧。

- **查询转换 (Transformation) 与扩张策略 (Expansion Strategy) :**
  - **Keyword Extraction :** 利用 NLP 工具提取 Query 中的关键实体与动词 (去停用词) , 生成用于稀疏检索的 Token 列表。
  - **\*\*Query Expansion\*\* :**
    - 系统可做 Synonym/Alias Expansion (同义词/别名/缩写扩展) , 默认策略采用“扩展融入稀疏检索、稠密检索保持单次”以控制成本与复杂度。
    - **Sparse Route (BM25) :** 将“关键词 + 同义词/别名”合并为一个查询表达式 (逻辑上按 OR 扩展) , 只执行一次稀疏检索。原始关键词可赋予更高权重以抑制语义漂移。
    - **Dense Route (Embedding) :** 使用原始 query (或轻度改写后的语义 query) 生成 embedding, 只执行一次稠密检索; 默认不为每个同义词单独触发额外的向量检索请求。
- **Hybrid Search Execution (双路混合检索)**
  - **并行召回 (Parallel Execution) :**
    - **Dense Route :** 计算 Query Embedding -> 检索向量库 (Cosine Similarity) -> 返回 Top-N 语义候选。
    - **Sparse Route :** 使用 BM25 算法 -> 检索倒排索引 -> 返回 Top-N 关键词候选。
  - **结果融合 (Fusion) :**
    - 采用 **RRF (Reciprocal Rank Fusion)** 算法, 不依赖各路分数的绝对值, 而是基于排名的倒数进行加权融合。
    - 公式策略:  $Score = 1 / (k + Rank\_Dense) + 1 / (k + Rank\_Sparse)$  , 平滑因单一模态缺陷导致的漏召回。
- **Filtering & Reranking (精确过滤与重排)**
  - **Metadata Filtering Strategy (通用过滤策略) :**
    - 原则: 先解析、能前置则前置、无法前置则后置兜底。
    - Query Processing 阶段应将结构化约束解析为通用 **filters** (例如 `collection/doc_type/language/time_range/access_level` 等)。
    - 若底层索引支持且属于硬约束 (Hard Filter) , 则在 Dense/Sparse 检索阶段做 Pre-filter 以缩小候选集、降低成本。
    - 无法前置的过滤 (索引不支持或字段缺失/质量不稳) 在 Rerank 前统一做 Post-filter 作为 safety net; 对缺失字段默认采取“宽松包含”(missing->include) 以避免误杀召回。
    - 软偏好 (Soft Preference, 例如“更近期更好”) 不应硬过滤, 而应作为排序信号在融合/重排阶段加权。
  - **Rerank Backend (可插拔精排后端) :**
    - 目标: 在 Top-M 候选上进行高精度排序/过滤; 该模块必须可关闭, 并提供稳定回退策略。
    - 后端选项:
      1. **None (关闭精排) :** 直接返回融合后的 Top-K (RRF 排名作为最终结果)。
      2. **Cross-Encoder Rerank (本地/托管模型) :** 输入为 `[Query, Chunk]` 对, 输出相关性分数并排序; 适合稳定、结构化输出。CPU 环境下建议默认仅对较小的 Top-M 执行 (例如  $M=10\sim30$ ) , 并提供超时回退。
      3. **LLM Rerank (可选) :** 使用 LLM 对候选集排序/选择; 适合需要更强指令理解或无本地模型环境时。为控制成本与稳定性, 候选数应更小 (例如  $M\leq 20$ ) , 并要求输出严格结构化格式 (如 JSON 的 ranked ids)。
    - 默认与回退 (Fallback) :



- 默认策略面向通用框架与 CPU 环境：优先保证“可用与可控”，Cross-Encoder/LLM 均为可选增强。
- 当精排不可用/超时/失败时，必须回退到融合阶段的排序（RRF Top-K），确保系统可用性与结果稳定性。

## 3.2 MCP 服务设计 (MCP Service Design)

**目标：**设计并实现一个符合 Model Context Protocol (MCP) 规范的 Server，使其能够作为知识上下文提供者，无缝对接主流 MCP Clients（如 GitHub Copilot、Claude Desktop 等），让用户通过现有 AI 助手即可查询私有知识库。

### 3.2.1 核心设计理念

- **协议优先 (Protocol-First)**：严格遵循 MCP 官方规范（JSON-RPC 2.0），确保与任何合规 Client 的互操作性。
- **开箱即用 (Zero-Config for Clients)**：Client 端无需任何特殊配置，只需在配置文件中添加 Server 连接信息即可使用全部功能。
- **引用透明 (Citation Transparency)**：所有检索结果必须携带完整的来源信息，支持 Client 端展示“回答依据”，增强用户对 AI 输出的信任。
- **多模态友好 (Multimodal-Ready)**：返回格式应支持文本与图像等多种内容类型，为未来的富媒体展示预留扩展空间。

### 3.2.2 传输协议：Stdio 本地通信

本项目采用 **Stdio Transport** 作为唯一通信模式。

- **工作方式**：Client（VS Code Copilot、Claude Desktop）以子进程方式启动我们的 Server，双方通过标准输入/输出交换 JSON-RPC 消息。
- **选型理由**：
  - **零配置**：无需网络端口、无需鉴权，用户只需在 Client 配置文件中指定启动命令即可使用。
  - **隐私安全**：数据不经过网络，天然适合处理私有知识库与敏感业务数据。
  - **契合定位**：Stdio 完美适配开发者本地工作流，满足私有知识管理与快速原型验证需求。
- **实现约束**：
  - **stdout** 仅输出合法 MCP 消息，禁止混入任何日志或调试信息。
  - 日志统一输出至 **stderr**，避免污染通信通道。

### 3.2.3 SDK 与实现库选型

- **首选：Python 官方 MCP SDK (mcp)**
  - **优势**：
    - 官方维护，与协议规范同步更新，保证最新特性支持（如 `outputSchema`、`annotations` 等）。
    - 提供 `@server.tool()` 等装饰器，声明式定义 Tools/Resources/Prompts，代码简洁。
    - 内置 Stdio 与 HTTP Transport 支持，无需手动处理 JSON-RPC 序列化与生命周期管理。
  - **适用**：本项目的默认实现方案。
- **备选：FastAPI + 自定义协议层**

- **场景**：需要深度定制 HTTP 行为（如自定义中间件、复杂鉴权流程）或希望学习 MCP 协议底层细节时可考虑。
- **权衡**：开发成本更高，需自行实现能力协商 (Capability Negotiation)、错误码映射等，且需持续跟进协议版本更新。
- **协议版本**：跟踪 MCP 最新稳定版本（如 2025-06-18），在 `initialize` 阶段进行版本协商，确保 Client/Server 兼容性。

3.2.4 对外暴露的工具函数设计 (Tools Design)

Server 通过 `tools/list` 向 Client 注册可调用的工具函数。工具设计应遵循"单一职责、参数明确、输出丰富"原则。

- **核心工具集**：

| 工具名称                              | 功能描述                          | 典型输入参数                                                                                   | 输出特点          |
|-----------------------------------|-------------------------------|------------------------------------------------------------------------------------------|---------------|
| <code>query_knowledge_hub</code>  | 主检索入口，执行混合检索 + Rerank，返回最相关片段 | <code>query: string</code> , <code>top_k?: int</code> , <code>collection?: string</code> | 返回带引用的结构化结果   |
| <code>list_collections</code>     | 列举知识库中可用的文档集合                 | 无                                                                                        | 集合名称、描述、文档数量  |
| <code>get_document_summary</code> | 获取指定文档的摘要与元信息                 | <code>doc_id: string</code>                                                              | 标题、摘要、创建时间、标签 |

- **扩展工具 (Agentic 演进方向)**：
  - `search_by_keyword` / `search_by_semantic`：拆分独立的检索策略，供 Agent 自主选择。
  - `verify_answer`：事实核查工具，检测生成内容是否有依据支撑。
  - `list_document_sections`：浏览文档目录结构，支持多步导航式检索。

3.2.5 返回内容与引用透明设计 (Response & Citation Design)

MCP 协议的 Tool 返回格式支持多种内容类型 (`content` 数组)，本项目将充分利用这一特性实现"可溯源"的回答：

- **结构化引用设计**：
  - 每个检索结果片段应包含完整的定位信息：`source_file` (文件名/路径)、`page` (页码，如适用)、`chunk_id` (片段标识)、`score` (相关性分数)。
  - 推荐在返回的 `structuredContent` 中采用统一的 Citation 格式：

```
{
  "answer": "...",
  "citations": [
    { "id": 1, "source": "xxx.pdf", "page": 5, "text": "原文片段...",
      "score": 0.92 },
    ...
  ]
}
```

```
]
}
```

- 同时在 `content` 数组中以 Markdown 格式呈现人类可读的带引用回答 ( [1] 标注 )，保证 Client 无论是否解析结构化内容都能展示引用。
- 多模态内容返回：
  - 文本内容 (**TextContent**)：默认返回类型，Markdown 格式，支持代码块、列表等富文本。
  - 图像内容 (**ImageContent**)：当检索结果关联图像时，Server 读取本地图片文件并编码为 Base64 返回。
    - 格式：`{ "type": "image", "data": "<base64>", "mimeType": "image/png" }`
    - 工作流程：数据摄取阶段存储图片本地路径 → 检索命中后 Server 动态读取 → 编码为 Base64 → 嵌入返回消息。
    - **Client 兼容性**：图像展示能力取决于 Client 实现，GitHub Copilot 可能降级处理，Claude Desktop 支持完整渲染。Server 端统一返回 Base64 格式，由 Client 决定如何渲染。
- **Client 适配策略**：
  - **GitHub Copilot (VS Code)**：当前对 MCP 的支持集中在 Tools 调用，返回的 `content` 中的文本会展示给用户。建议以清晰的 Markdown 文本 (含引用标注) 为主，图像作为补充。
  - **Claude Desktop**：对 MCP Tools/Resources 有完整支持，图像与资源链接可直接渲染。可更激进地使用多模态返回。
  - **通用兼容原则**：始终在 `content` 数组第一项提供纯文本/Markdown 版本的答案，确保最低兼容性；将结构化数据、图像等放在后续项或 `structuredContent` 中，供高级 Client 解析。

### 3.3 可插拔架构设计 (Pluggable Architecture Design)

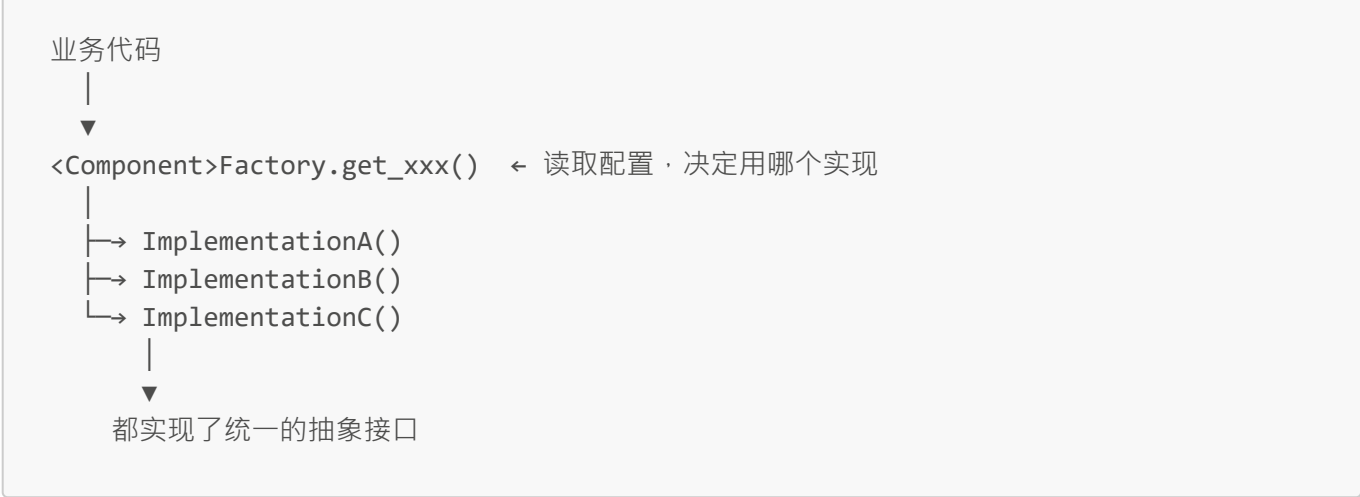
**目标**：定义清晰的抽象层与接口契约，使 RAG 链路的每个核心组件都能够独立替换与升级，避免技术锁定，支持低成本的 A/B 测试与环境迁移。

**术语说明**：本节中的“提供者 (Provider)”、“实现 (Implementation)”指的是完成某项功能的具体技术方案，而非传统 Web 架构中的“后端服务器”。例如，LLM 提供者可以是远程的 Azure OpenAI API，也可以是本地运行的 Ollama；向量存储可以是本地嵌入式的 Chroma，也可以是云端托管的 Pinecone。本项目作为本地 MCP Server，通过统一接口对接这些不同的提供者，实现灵活切换。

#### 3.3.1 设计原则

- **接口隔离 (Interface Segregation)**：为每类组件定义最小化的抽象接口，上层业务逻辑仅依赖接口而非具体实现。
- **配置驱动 (Configuration-Driven)**：通过统一配置文件 (如 `settings.yaml`) 指定各组件的具体后端，代码无需修改即可切换实现。
- **工厂模式 (Factory Pattern)**：使用工厂函数根据配置动态实例化对应的实现类，实现“一处配置，处处生效”。
- **优雅降级 (Graceful Fallback)**：当首选后端不可用时，系统应自动回退到备选方案或安全默认值，保障可用性。

**通用结构示意 (适用于 3.3.2 / 3.3.3 / 3.3.4 等可插拔组件)：**



3.3.2 LLM 与 Embedding 提供者抽象

这是可插拔设计的核心环节，因为模型提供者的选择直接影响成本、性能与隐私合规。

- 统一接口层 (Unified API Abstraction) :
  - 设计思路：无论底层使用 Azure OpenAI、OpenAI 原生 API、DeepSeek 还是本地 Ollama，上层调用代码应保持一致。
  - 关键抽象：
    - LLMClient：暴露 chat(messages) -> response 方法，屏蔽不同 Provider 的认证方式与请求格式差异。
    - EmbeddingClient：暴露 embed(texts) -> vectors 方法，统一处理批量请求与维度归一化。
- 提供者选项与切换场景：

| 提供者类型              | 典型场景               | 配置切换点                                               |
|--------------------|--------------------|-----------------------------------------------------|
| Azure OpenAI       | 企业合规、私有云部署、区域数据驻留  | provider: azure, endpoint, api_key, deployment_name |
| OpenAI 原生          | 通用开发、最新模型尝鲜        | provider: openai, api_key, model                    |
| DeepSeek / 其他云端    | 成本优化、特定语言优化        | provider: deepseek, api_key, model                  |
| Ollama / vLLM (本地) | 完全离线、隐私敏感、无 API 成本 | provider: ollama, base_url, model                   |

- 技术选型建议：
  - 如 3.1 节所述，本项目以 LlamaIndex 为主框架，其内置了对主流 LLM/Embedding Provider 的适配 (OpenAI、Azure、Ollama 等)。LlamaIndex 的 LLM 和 Embedding 抽象类已封装了统一调用接口。
  - 对于 LlamaIndex 未覆盖的 Provider (如 DeepSeek)，可通过其 OpenAI-Compatible 模式接入 (设置自定义 api\_base)，或引入 LangChain 的对应适配器。
  - 对于企业级需求，可在其基础上增加统一的重试、限流、日志中间层，提升生产可靠性，但本项目暂不实现，这里仅提供思路。

3.3.3 检索策略抽象

检索层的可插拔性决定了系统在不同数据规模与查询模式下的适应能力。

设计模式：抽象工厂模式

与 3.3.2 节的 LLM 抽象类似，检索层各组件的可插拔性同样依赖两层设计：

- 1. 框架提供的统一接口：本项目采用 **LlamaIndex Ingestion Pipeline** 作为核心框架，其为向量数据库、Embedding 等组件定义了统一的抽象接口，不同实现只需遵循相同接口即可无缝替换。
- 2. 我们编写的工厂函数：对于框架未覆盖的组件（如稀疏检索、融合策略），我们自行定义抽象接口并编写工厂函数，根据配置决定实例化哪个具体实现。

通用的“配置驱动 + 工厂路由”结构示意见 3.3.1 节。

下面分别说明各组件如何应用这一模式：

1. 分块策略 (Chunking Strategy)

分块是 Ingestion Pipeline 的核心环节之一，决定了文档如何被切分为适合检索的语义单元。LlamaIndex Ingestion Pipeline 的 Splitter 环节支持可插拔设计，不同分块实现只需遵循相同接口即可无缝替换。

常见的分块策略包括：

- 固定长度切分：按字符数或 Token 数切分，简单但可能破坏语义完整性。
- 递归字符切分：按层级分隔符（段落→句子→字符）递归切分，在长度限制内尽量保持语义边界。
- 语义切分：利用 Embedding 相似度检测语义断点，确保每个 Chunk 是自包含的语义单元。
- 结构感知切分：根据文档结构（Markdown 标题、代码块、列表等）进行切分。

本项目当前采用 **LangChain** 的 **RecursiveCharacterTextSplitter** 进行切分，该方法对 Markdown 文档的结构（标题、段落、列表、代码块）有天然的适配性，能够通过配置语义断点（Separators）实现高质量、语义完整的切块。

**当前实现说明：**目前系统使用 LangChain RecursiveCharacterTextSplitter。架构设计上预留了切换能力，如需使用 LlamaIndex 的 SentenceSplitter、SemanticSplitter 或自定义切分器，可在 Pipeline 中替换相应组件。

2. 向量数据库 (Vector Store)

LlamaIndex 为向量数据库定义了统一的 **VectorStore** 抽象接口，所有主流向量库（Chroma、Qdrant、Pinecone 等）都有对应适配器，暴露相同的 **.add()**、**.query()** 等方法。我们通过 **VectorStoreFactory** 根据配置选择具体实现。

本项目选用 **Chroma** 作为向量数据库。相比 Qdrant、Milvus、Weaviate 等需要 Docker 容器或分布式架构支撑的方案，Chroma 采用嵌入式设计，**pip install chromadb** 即可使用，无需额外部署数据库服务，非常适合本地开发与快速原型验证。同时 LlamaIndex 提供了成熟的 **ChromaVectorStore** 适配器，与 Ingestion Pipeline 无缝集成。

**当前实现说明：**目前系统仅实现了 Chroma 后端。虽然架构设计上预留了工厂模式以支持未来扩展，但当前版本尚未实现其他向量数据库的适配器。

### 3. 向量编码策略 (Embedding Strategy)

向量编码是 Ingestion Pipeline 的关键环节，决定了 Chunk 如何被转换为可检索的向量表示。LlamaIndex 提供了 **BaseEmbedding** 抽象接口，支持不同 Embedding 模型的可插拔替换。

常见的编码策略包括：

- **纯稠密编码 (Dense Only)**：仅生成语义向量，适合通用场景。
- **纯稀疏编码 (Sparse Only)**：仅生成关键词权重向量，适合精确匹配场景。
- **双路编码 (Dense + Sparse)**：同时生成稠密向量和稀疏向量，为混合检索提供数据基础。

本项目当前采用 **双路编码 (Dense + Sparse)** 策略：

- **Dense Embeddings (语义向量)**：调用 Embedding 模型（如 OpenAI text-embedding-3）生成高维浮点向量，捕捉文本的深层语义关联。
- **Sparse Embeddings (稀疏向量)**：利用 BM25 编码器生成稀疏向量（Keyword Weights），捕捉精确的关键词匹配信息。

存储时，Dense Vector 和 Sparse Vector 与 Chunk 原文、Metadata 一起原子化写入向量数据库，确保检索时可同时利用两种向量。

**当前实现说明：**目前系统实现了 Dense + Sparse 双路编码。架构设计上预留了切换能力，如需使用其他 Embedding 模型（如 BGE、Ollama 本地模型）或调整编码策略，可在 Pipeline 中替换相应组件。

### 4. 召回策略 (Retrieval Strategy)

召回策略决定了查询阶段如何从知识库中检索相关内容。基于 Ingestion 阶段存储的向量类型，可采用不同的召回方案：

- **纯稠密召回 (Dense Only)**：仅使用语义向量进行相似度匹配。
- **纯稀疏召回 (Sparse Only)**：仅使用 BM25 进行关键词匹配。
- **混合召回 (Hybrid)**：并行执行稠密和稀疏两路召回，再通过融合算法合并结果。
- **混合召回 + 精排 (Hybrid + Rerank)**：在混合召回基础上，增加精排步骤进一步提升相关性。

本项目当前采用 **混合召回 + 精排 (Hybrid + Rerank)** 策略：

- **稠密召回 (Dense Route)**：计算 Query Embedding，在向量库中进行 Cosine Similarity 检索，返回 Top-N 语义候选。
- **稀疏召回 (Sparse Route)**：使用 BM25 算法检索倒排索引，返回 Top-N 关键词候选。
- **融合 (Fusion)**：使用 RRF (Reciprocal Rank Fusion) 算法将两路结果合并排序。
- **精排 (Rerank)**：对融合后的候选集进行重排序，支持 None / Cross-Encoder / LLM Rerank 三种模式。

**当前实现说明：**目前系统实现了 Hybrid + Rerank 策略。架构设计上预留了策略切换能力，如需使用纯稠密或纯稀疏召回，可通过配置切换；融合算法和 Reranker 同样支持替换。

#### 3.3.4 评估框架抽象

评估体系的可插拔性确保团队可以根据业务目标灵活选择或组合不同的质量度量维度。

- **设计思路：**



- 定义统一的 `Evaluator` 接口，暴露 `evaluate(query, retrieved_chunks, generated_answer, ground_truth) -> metrics` 方法。
- 各评估框架实现该接口，输出标准化的指标字典。

• 可选评估框架：

| 框架       | 特点                                                                  | 适用场景                    |
|----------|---------------------------------------------------------------------|-------------------------|
| Ragas    | RAG 专用、指标丰富 ( Faithfulness, Answer Relevancy, Context Precision 等 ) | 全面评估 RAG 质量、学术对比        |
| DeepEval | LLM-as-Judge 模式、支持自定义评估标准                                           | 需要主观质量判断、复杂业务规则         |
| 自定义指标    | Hit Rate, MRR, Latency P99 等基础工程指标                                  | 快速回归测试、上线前 Sanity Check |

- 组合与扩展：
  - 评估模块设计为 `组合模式`，可同时挂载多个 `Evaluator`，生成综合报告。
  - 配置示例：`evaluation.backends: [ragas, custom_metrics]`，系统并行执行并汇总结果。

3.3.5 配置管理与切换流程

- 配置文件结构示例 (`config/settings.yaml`)：

```
llm:
  provider: azure # azure | openai | ollama | deepseek
  model: gpt-4o
  # provider-specific configs...

embedding:
  provider: openai
  model: text-embedding-3-small

vector_store:
  backend: chroma # chroma | qdrant | pinecone

retrieval:
  sparse_backend: bm25 # bm25 | elasticsearch
  fusion_algorithm: rrf # rrf | weighted_sum
  rerank_backend: cross_encoder # none | cross_encoder | llm

evaluation:
  backends: [ragas, custom_metrics]
```

- 切换流程：
  1. 修改 `settings.yaml` 中对应组件的 `backend` / `provider` 字段。
  2. 确保新后端的依赖已安装、凭据已配置。
  3. 重启服务，工厂函数自动加载新实现，无需修改业务代码。

3.4 可观测性与追踪设计 (Observability & Tracing Design)

目标：针对 RAG 系统常见的"黑盒"问题，设计全链路可观测的追踪体系，使每一次检索与生成过程都透明可见且可量化，为调试优化与质量评估提供数据基础。

3.4.1 设计理念

- 请求级全链路追踪 (Request-Level Tracing)：以 `trace_id` 为核心，完整记录单次请求从 Query 输入到 Response 输出的全过程，包括各阶段的输入输出、耗时与评估分数。
- 透明可回溯 (Transparent & Traceable)：每个阶段的中间状态都被记录，开发者可以清晰看到"系统为什么召回了这些文档"、"Rerank 前后排名如何变化"，从而精准定位问题。
- 低侵入性 (Low Intrusiveness)：追踪逻辑与业务逻辑解耦，通过装饰器或回调机制注入，避免污染核心代码。
- 轻量本地化 (Lightweight & Local)：采用结构化日志 + 本地 Dashboard 的方案，零外部依赖，开箱即用。

3.4.2 追踪数据结构

每次用户请求生成唯一的 `trace_id`，作为日志关联与问题排查的核心标识。一条 Trace 记录包含以下信息：

基础信息：

- `trace_id`：请求唯一标识
- `timestamp`：请求时间戳
- `user_query`：用户原始查询
- `collection`：检索的知识库集合

各阶段详情 (Stages)：

每个阶段记录其输入、输出、耗时与关键指标：

| 阶段               | 记录内容                              |
|------------------|-----------------------------------|
| Query Processing | 原始 Query、改写后 Query (若有)、提取的关键词、耗时 |
| Dense Retrieval  | 返回的 Top-N 候选及相似度分数、耗时             |
| Sparse Retrieval | 返回的 Top-N 候选及 BM25 分数、耗时          |
| Fusion           | 融合后的统一排名、耗时                       |
| Rerank           | 重排后的最终排名及分数、是否触发 Fallback、耗时      |

汇总指标：

- `total_latency`：端到端总耗时
- `top_k_results`：最终返回的 Top-K 文档 ID
- `error`：异常信息 (若有)

评估指标 (Evaluation Metrics)：

每次请求可选计算轻量级评估分数，直接记录在 Trace 中，便于回溯分析：

- `context_relevance`：召回文档与 Query 的相关性分数
- `answer_faithfulness`：生成答案与召回文档的一致性分数（若有生成环节）

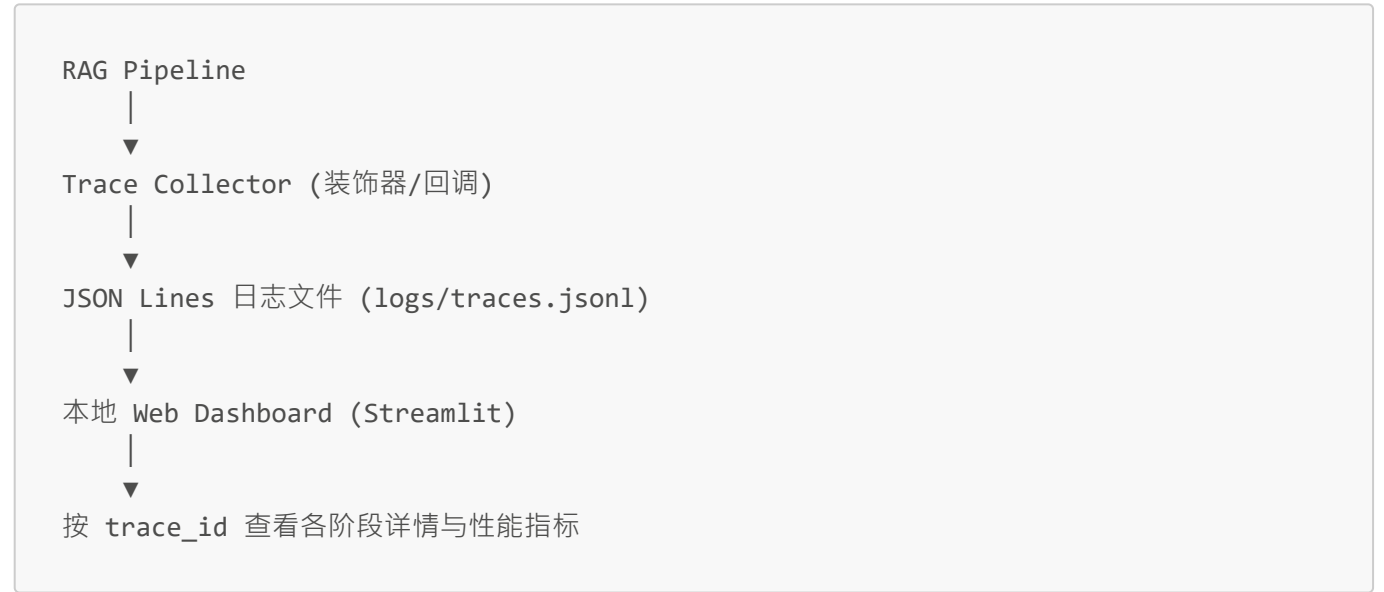
### 3.4.3 技术方案：结构化日志 + 本地 Web Dashboard

本项目采用 "结构化日志 + 本地 Web Dashboard" 作为可观测性的实现方案。

选型理由：

- **零外部依赖**：不依赖 LangSmith、LangFuse 等第三方平台，无需网络连接与账号注册，完全本地化运行。
- **轻量易部署**：仅需 Python 标准库 + 一个轻量 Web 框架（如 Streamlit），`pip install` 即可使用，无需 Docker 或数据库服务。
- **学习成本低**：结构化日志是通用技能，调试时可直接用 `jq`、`grep` 等命令行工具查询；Dashboard 代码简单直观，便于理解与二次开发。
- **契合项目定位**：本项目面向本地 MCP Server 场景，单用户、单机运行，无需分布式追踪或多租户隔离等企业级能力。

实现架构：



核心组件：

- **结构化日志层**：基于 Python `logging` + JSON Formatter，将每次请求的 Trace 数据以 JSON Lines 格式追加写入本地文件。每行一条完整的请求记录，包含 `trace_id`、各阶段详情与耗时。
- **本地 Web Dashboard**：基于 Streamlit 构建的轻量级 Web UI，读取日志文件并提供交互式可视化。核心功能是按 `trace_id` 检索并展示单次请求的完整追踪链路。

### 3.4.4 追踪机制实现

为确保各 RAG 阶段（可替换、可自定义）都能输出统一格式的追踪日志，系统采用 **TraceContext**（追踪上下文）作为核心机制。

工作原理：

1. **请求开始**：Pipeline 入口创建一个 `TraceContext` 实例，生成唯一 `trace_id`，记录请求基础信息（Query、Collection 等）。
2. **阶段记录**：`TraceContext` 提供 `record_stage()` 方法，各阶段执行完毕后调用该方法，传入阶段名称、耗时、输入输出等数据。
3. **请求结束**：调用 `trace.finish()`，`TraceContext` 将收集的完整数据序列化为 JSON，追加写入日志文件。

与可插拔组件的配合：

- 各阶段组件（Retriever、Reranker 等）的接口约定中包含 `TraceContext` 参数。
- 组件实现者在执行核心逻辑后，调用 `trace.record_stage()` 记录本阶段的关键信息。
- 这是**显式调用**模式：不强制、不会因未调用而报错，但依赖开发者主动记录。好处是代码透明，开发者清楚知道哪些数据被记录；代价是需要开发者自觉遵守约定。

阶段划分原则：

- **Stage 是固定的通用大类**：`retrieval`（检索）、`rerank`（重排）、`generation`（生成）等，不随具体实现方案变化。
- **具体实现是阶段内部的细节**：在 `record_stage()` 中通过 `method` 字段记录采用的具体方法（如 `bm25`、`hybrid`），通过 `details` 字段记录方法相关的细节数据。
- 这样无论底层方案怎么替换，阶段结构保持稳定，Dashboard 展示逻辑无需调整。

### 3.4.5 Dashboard 功能

Dashboard 以 `trace_id` 为核心，提供以下视图：

- **请求列表**：按时间倒序展示历史请求，支持按 Query 关键词筛选。
- **单请求详情页**：
  - **耗时瀑布图**：展示各阶段的时间分布，快速定位性能瓶颈。
  - **阶段详情展开**：点击任意阶段，查看该阶段的输入、输出与关键参数。
  - **召回结果表**：展示 Top-K 候选文档在各阶段的排名与分数变化。

### 3.4.6 配置示例

```
observability:
  enabled: true

# 日志配置
logging:
  log_file: logs/traces.jsonl # JSON Lines 格式日志文件
  log_level: INFO # DEBUG | INFO | WARNING

# 追踪粒度控制
detail_level: standard # minimal | standard | verbose

# Dashboard 配置
dashboard:
```

```
enabled: true
port: 8501 # Streamlit 默认端口
```

### 3.5 多模态图片处理设计 (Multimodal Image Processing Design)

**目标：**设计一套完整的图片处理方案，使 RAG 系统能够理解、索引并检索文档中的图片内容，实现“用自然语言搜索图片”的能力，同时保持架构的简洁性与可扩展性。

#### 3.5.1 设计理念与策略选型

多模态 RAG 的核心挑战在于：**如何让纯文本的检索系统“看懂”图片**。业界主要有两种技术路线：

| 策略                             | 核心思路                                  | 优势             | 劣势                     |
|--------------------------------|---------------------------------------|----------------|------------------------|
| <b>Image-to-Text (图转文)</b>     | 利用 Vision LLM 将图片转化为文本描述，复用纯文本 RAG 链路 | 架构统一、实现简单、成本可控 | 描述质量依赖 LLM 能力，可能丢失视觉细节 |
| <b>Multi-Embedding (多模态向量)</b> | 使用 CLIP 等模型将图文统一映射到同一向量空间             | 保留原始视觉特征，支持图搜图 | 需引入额外向量库，架构复杂度高        |

本项目选型：**Image-to-Text (图转文) 策略**

选型理由：

- **架构统一：**无需引入 CLIP 等多模态 Embedding 模型，无需维护独立的图像向量库，完全复用现有的文本 RAG 链路 (Ingestion → Hybrid Search → Rerank)。
- **语义对齐：**通过 LLM 将图片的视觉信息转化为自然语言描述，天然与用户的文本查询在同一语义空间，检索效果可预期。
- **成本可控：**仅在数据摄取阶段一次性调用 Vision LLM，检索阶段无额外成本。
- **渐进增强：**未来如需支持“图搜图”等高级能力，可在此基础上叠加 CLIP Embedding，无需重构核心链路。

#### 3.5.2 图片处理全流程设计

图片处理贯穿 Ingestion Pipeline 的多个阶段，整体流程如下：





3.5.3 各阶段技术要点

1. Loader 阶段：图片提取与引用收集

- 提取策略：
  - 解析文档时识别嵌入的图片资源（PDF 中的 XObject、PPT 中的媒体文件、Markdown 中的 `![]()` 引用）。
  - 为每张图片生成全局唯一的 `image_id`（建议格式：`{doc_hash}_{page}_{seq}`）。
  - 将图片二进制数据提取并暂存，记录其在原文档中的位置信息。
- 引用标记：
  - 在转换后的 Markdown 文本中，于图片原始位置插入占位符（如 `[IMAGE: {image_id}]`）。
  - 在 Document 的 Metadata 中维护 `images` 列表，记录每张图片的 `image_id`、原始路径、页码、尺寸等基础信息。
- 存储原始图片：
  - 将提取的图片保存至本地文件系统的约定目录（如 `data/images/{collection}/{image_id}.png`）。
  - 仅保存需要的图片格式（推荐统一转换为 PNG/JPEG），控制存储体积。

2. Splitter 阶段：保持图文关联

- 关联保持原则：
  - 图片引用标记应与其说明性文字（Caption、前后段落）尽量保持在同一 **Chunk** 中。
  - 若图片出现在章节开头或结尾，切分时应将其归入语义上最相关的 **Chunk**。
- **Chunk Metadata** 扩展：



- 每个 Chunk 的 Metadata 中增加 `image_refs: List[image_id]` 字段，记录该 Chunk 关联的图片列表。
- 此字段用于后续 Transform 阶段定位需要处理的图片，以及检索命中后定位需要返回的图片。

3. Transform 阶段：图片理解与描述生成

这是多模态处理的核心环节，负责将视觉信息转化为可检索的文本语义。

• Vision LLM 选型：

| 模型                | 提供商             | 特点                        | 适用场景               | 推荐指数  |
|-------------------|-----------------|---------------------------|--------------------|-------|
| GPT-4o            | OpenAI / Azure  | 理解能力强，支持复杂图表解读，英文文档表现优异   | 高质量需求、复杂业务文档、国际化场景 | ★★★★★ |
| Qwen-VL-Max       | 阿里云 (DashScope) | 中文理解能力出色，性价比高，对中文图表/文档支持好 | 中文文档、国内部署、成本敏感场景   | ★★★★★ |
| Qwen-VL-Plus      | 阿里云 (DashScope) | 速度更快，成本更低，适合大批量处理         | 大批量中文文档、快速迭代场景     | ★★★★  |
| Claude 3.5 Sonnet | Anthropic       | 多模态原生支持，长上下文              | 需要结合大段文字理解图片       | ★★★★  |
| Gemini Pro Vision | Google          | 成本较低，速度较快                 | 大批量处理、成本敏感场景       | ★★★   |
| GLM-4V            | 智谱 AI (ZhipuAI) | 国内老牌，稳定性好，中文支持佳           | 国内部署备选、企业级应用       | ★★★★  |

双模型选型策略（推荐）：

本项目采用国内 + 国外双模型方案，通过配置切换，兼顾不同部署环境和文档类型：

| 部署环境           | 主选模型           | 备选模型              | 说明                             |
|----------------|----------------|-------------------|--------------------------------|
| 国际化 / Azure 环境 | GPT-4o (Azure) | Qwen-VL-Max       | 英文文档优先用 GPT-4o，中文文档可切换 Qwen-VL |
| 国内部署 / 纯中文场景   | Qwen-VL-Max    | GPT-4o            | 中文图表理解用 Qwen-VL，特殊需求可切换 GPT-4o |
| 成本敏感 / 大批量     | Qwen-VL-Plus   | Gemini Pro Vision | 牺牲部分质量换取速度和成本                  |

选型理由：

1. GPT-4o (国外首选)：

- 视觉理解能力业界领先，复杂图表解读准确率高
- Azure 部署可满足企业合规要求
- 英文技术文档理解效果最佳

2. Qwen-VL-Max (国内首选)：

- 中文场景下表现与 GPT-4o 接近，部分中文图表任务甚至更优
- 通过阿里云 DashScope API 调用，国内访问稳定、延迟低
- 价格约为 GPT-4o 的 1/3 ~ 1/5，性价比极高
- 原生支持中文 OCR，对中文截图、表格识别更准确
- 描述生成策略：
  - **结构化 Prompt**：设计专用的图片理解 Prompt，引导 LLM 输出结构化描述，而非自由发挥。
  - **上下文感知**：将图片的前后文本段落一并传入 Vision LLM，帮助其理解图片在文档中的语境与作用。
  - **分类型处理**：针对不同类型的图片采用差异化的理解策略：

| 图片类型    | 理解重点           | Prompt 引导方向       |
|---------|----------------|-------------------|
| 流程图/架构图 | 节点、连接关系、流程逻辑   | "描述这张图的结构和流程步骤"   |
| 数据图表    | 数据趋势、关键数值、对比关系 | "提取图表中的关键数据和结论"   |
| 截图/UI   | 界面元素、操作指引、状态信息 | "描述截图中的界面内容和关键信息" |
| 照片/插图   | 主体对象、场景、视觉特征   | "描述图片中的主要内容"      |

- 描述注入方式：
  - **推荐：注入正文**：将生成的描述直接替换或追加到 Chunk 正文中的图片占位符位置，格式如 `[图片描述: {caption}]`。这样描述会被 Embedding 覆盖，可被直接检索。
  - **备选：注入 Metadata**：将描述存入 `chunk.metadata.image_captions` 字段。需确保检索时该字段也被索引。
- 幂等与增量处理：
  - 为每张图片的描述计算内容哈希，存入 `processing_cache` 表。
  - 重复处理时，若图片内容未变且 Prompt 版本一致，直接复用缓存的描述，避免重复调用 Vision LLM。

4. Storage 阶段：双轨存储

- 向量库存储（用于检索）：
  - 存储增强后的 Chunk，其正文已包含图片描述，Metadata 包含 `image_refs` 列表。
  - 检索时通过文本相似度即可命中包含相关图片描述的 Chunk。
- 原始图片存储（用于返回）：
  - 图片文件存储于本地文件系统，路径记录在独立的 `images` 索引表中。
  - 索引表字段：`image_id, file_path, source_doc, page, width, height, mime_type`。
  - 检索命中后，根据 Chunk 的 `image_refs` 查询索引表，获取图片文件路径用于返回。

3.5.4 检索与返回流程

当用户查询命中包含图片的 Chunk 时，系统需要将图片与文本一并返回：



**MCP 响应格式：**

```
{
  "content": [
    {
      "type": "text",
      "text": "根据文档，系统架构如下：...\n\n[1] 来源：architecture.pdf，第5页"
    },
    {
      "type": "image",
      "data": "<base64-encoded-image>",
      "mimeType": "image/png"
    }
  ]
}
```

**3.5.5 质量保障与边界处理**

- **描述质量检测：**
  - 对生成的描述进行基础质量检查（长度、是否包含关键信息）。
  - 若描述过短或 LLM 返回"无法识别"，标记该图片为 **low\_quality**，可选择人工复核或跳过索引。
- **大尺寸/特殊图片处理：**
  - 超大图片在传入 Vision LLM 前进行压缩（保持宽高比，限制最大边长）。
  - 对于纯装饰性图片（如分隔线、背景图），可通过尺寸或位置规则过滤，不进入描述生成流程。
- **批量处理优化：**
  - 图片描述生成支持批量异步调用，提高吞吐量。
  - 单个文档处理失败时，记录失败的图片 ID，不影响其他图片的处理进度。

- 降级策略：
  - 当 Vision LLM 不可用时，系统回退到"仅保留图片占位符"模式，图片不参与检索但不阻塞 Ingestion 流程。
  - 在 Chunk 中标记 `has_unprocessed_images: true`，后续可增量补充描述。

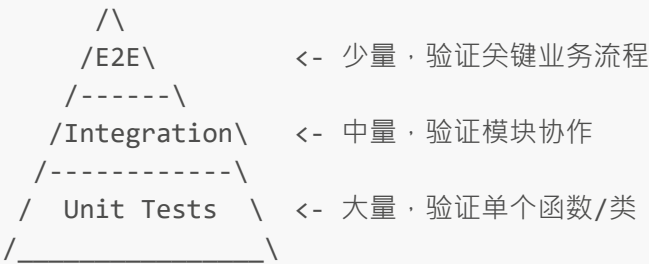
4. 测试方案

4.1 设计理念：测试驱动开发 (TDD)

本项目采用\*\*测试驱动开发（Test-Driven Development）\*\*作为核心开发范式，确保每个组件在实现前就已明确其预期行为，通过自动化测试持续验证系统质量。

核心原则：

- 早测试、常测试**：每个功能模块实现的同时就编写对应的单元测试，而非事后补测。
- 测试即文档**：测试用例本身就是最准确的行为规范，新加入的开发者可通过阅读测试快速理解各模块功能。
- 快速反馈循环**：单元测试应在秒级完成，支持开发者高频执行，立即发现引入的问题。
- 分层测试金字塔**：大量快速的单元测试作为基座，少量关键路径的集成测试作为保障，极少数端到端测试验证完整流程。



4.2 测试分层策略

4.2.1 单元测试 (Unit Tests)

目标：验证每个独立组件的内部逻辑正确性，隔离外部依赖。

覆盖范围：

| 模块             | 测试重点              | 典型测试用例                                 |
|----------------|-------------------|----------------------------------------|
| Loader (文档解析器) | 格式解析、元数据提取、图片引用收集 | - 测试解析单页/多页 PDF                        |
|                |                   | - 验证 Markdown 标题层级提取                   |
| Splitter (切分器) | 切分边界、上下文保留、元数据传递  | - 检查图片占位符插入位置                          |
|                |                   | - 验证按标题切分不破坏段落                         |
|                |                   | - 测试超长文本的递归切分                          |
|                |                   | - 检查 Chunk 的 <code>source</code> 字段正确性 |

| 模块              | 测试重点          | 典型测试用例                                                               |
|-----------------|---------------|----------------------------------------------------------------------|
| Transform (增强器) | 图片描述生成、元数据注入  | - Mock Vision LLM，验证描述注入逻辑<br>- 测试无图片时的降级行为<br>- 验证幂等性（重复处理相同输入）     |
| Embedding (向量化) | 批处理、差量计算、向量维度 | - 验证相同文本生成相同向量<br>- 测试批量请求的拆分与合并<br>- 检查缓存命中逻辑                       |
| BM25 (稀疏编码)     | 关键词提取、权重计算    | - 验证停用词过滤<br>- 测试 IDF 计算准确性<br>- 检查稀疏向量格式                            |
| Retrieval (检索器) | 召回精度、融合算法     | - 测试纯 Dense/Sparse/Hybrid 三种模式<br>- 验证 RRF 融合分数计算<br>- 检查 Top-K 结果排序 |
| Reranker (重排器)  | 分数归一化、降级回退    | - Mock Cross-Encoder，验证分数重排<br>- 测试超时后的 Fallback 逻辑<br>- 验证空候选集处理    |

技术选型：

- 测试框架：`pytest`（Python 标准选择，支持参数化测试、Fixture 机制）
- Mock 工具：`unittest.mock` / `pytest-mock`（隔离外部依赖，如 LLM API）
- 断言增强：`pytest-check`（支持多断言不中断执行）

4.2.2 集成测试 (Integration Tests)

目标：验证多个组件协作时的数据流转与接口兼容性。

覆盖范围：

| 测试场景               | 验证要点                                          | 测试策略                                                                                       |
|--------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------|
| Ingestion Pipeline | Loader → Splitter → Transform → Storage 的完整流程 | - 使用真实的测试 PDF 文件<br>- 验证最终存入向量库的数据完整性<br>- 检查中间产物（如临时图片文件）是否正确清理                           |
| Hybrid Search      | Dense + Sparse 召回的融合结果                        | - 准备已知答案的查询-文档对<br>- 验证融合后的 Top-1 是否命中正确文档<br>- 测试极端情况（某一路无结果）                             |
| Rerank Pipeline    | 召回 → 过滤 → 重排的组合                               | - 验证 Metadata 过滤后的候选集正确性<br>- 检查 Reranker 是否改变了 Top-1 结果<br>- 测试 Reranker 失败时的回退           |
| MCP Server         | 工具调用的端到端流程                                    | - 模拟 MCP Client 发送 JSON-RPC 请求<br>- 验证返回的 <code>content</code> 格式符合协议<br>- 测试错误处理（如查询语法错误） |

技术选型：

- 数据隔离：每个测试使用独立的临时数据库/向量库 ( `pytest-tempdir` )
- 异步测试： `pytest-asyncio` ( 若 MCP Server 采用异步实现 )
- 契约测试：定义各模块间的 Schema，确保接口不漂移

#### 4.2.3 端到端测试 (End-to-End Tests)

目标：模拟真实用户操作，验证完整业务流程的可用性。

核心场景：

##### 场景 1：数据准备 ( 离线摄取 )

- 测试目标：验证文档摄取流程的完整性与正确性
- 测试步骤：
  - 准备测试文档 ( PDF 文件，包含文本、图片、表格等多种元素 )
  - 执行离线摄取脚本，将文档导入知识库
  - 验证摄取结果：检查生成的 Chunk 数量、元数据完整性、图片描述生成
  - 验证存储状态：确认向量库和 BM25 索引正确创建
  - 验证幂等性：重复摄取同一文档，确保不产生重复数据
- 验证要点：
  - Chunk 的切分质量 ( 语义完整性、上下文保留 )
  - 元数据字段完整性 ( source、page、title、tags 等 )
  - 图片处理结果 ( Caption 生成、Base64 编码存储 )
  - 向量与稀疏索引的正确性

##### 场景 2：召回测试

- 测试目标：验证检索系统的召回精度与排序质量
- 测试步骤：
  - 基于已摄取的知识库，准备一组测试查询 ( 包含不同难度与类型 )
  - 执行混合检索 ( Dense + Sparse + Rerank )
  - 验证召回结果：检查 Top-K 文档是否包含预期来源
  - 对比不同检索策略的效果 ( 纯 Dense、纯 Sparse、Hybrid )
  - 验证 Rerank 的影响：对比重排前后的结果变化
- 验证要点：
  - Hit Rate@K：Top-K 结果命中率是否达标
  - 排序质量：正确答案是否排在前列 ( MRR、NDCG )
  - 边界情况处理：空查询、无结果查询、超长查询
  - 多模态召回：包含图片的文档是否能通过文本查询召回

##### 场景 3：MCP Client 功能测试

- 测试目标：验证 MCP Server 与 Client ( 如 GitHub Copilot ) 的协议兼容性与功能完整性
- 测试步骤：
  - 启动 MCP Server ( Stdio Transport 模式 )
  - 模拟 MCP Client 发送各类 JSON-RPC 请求
  - 测试工具调用： `query_knowledge_hub`、`list_collections` 等

- 验证返回格式：符合 MCP 协议规范 ( content 数组、structuredContent)
- 测试引用透明性：返回结果包含完整的 Citation 信息
- 测试多模态返回：包含图片的响应正确编码为 Base64
- 验证要点：
  - 协议合规性：JSON-RPC 2.0 格式、错误码映射
  - 工具注册：tools/list 返回所有可用工具及其 Schema
  - 响应格式：TextContent 与 ImageContent 的正确组合
  - 错误处理：无效参数、超时、服务不可用等异常场景
  - 性能指标：单次请求的端到端延迟 ( 含检索、重排、格式化 )

测试工具：

- BDD 框架：behave 或 pytest-bdd ( 以 Gherkin 语法描述场景 )
- 环境准备：
  - 临时测试向量库 ( 独立于生产数据 )
  - 预置的标准测试文档集
  - 本地 MCP Server 进程 ( Stdio Transport)

4.3 RAG 质量评估测试

目标：验证已设计的评估体系 ( 见 3.3.4 评估框架抽象 ) 是否正确实现，并能有效评估 RAG 系统的召回与生成质量。

测试要点：

1. 黄金测试集准备

- 构建标准的"问题-答案-来源文档"测试集 ( JSON 格式 )
- 初期人工标注核心场景，后期持续积累坏 Case

2. 评估框架实现验证

- 验证 Ragas/DeepEval 等评估框架的正确集成
- 确认评估接口能输出标准化的指标字典
- 测试多评估器并行执行与结果汇总

3. 关键指标达标验证

- 检索指标：Hit Rate@K ≥ 90%、MRR ≥ 0.8、NDCG@K ≥ 0.85
- 生成指标：Faithfulness ≥ 0.9、Answer Relevancy ≥ 0.85
- 定期运行评估，监控指标是否回归

说明：本节重点是验证评估体系的工程实现，而非重新设计评估方法 ( 评估方法的设计见第 3 章技术选型 )。

4.4 性能与压力测试 ( 可选 )

说明：本项目定位为本地 MCP Server，单用户开发环境，采用 Stdio Transport 通信方式。性能与压力测试在当前阶段不是必需的，此处列出主要用于：

1. 架构完整性：展示完整的工程化测试体系，体现系统设计的专业性

2. 未来扩展性：若后续需要云端部署或多用户支持，可直接参考此方案



3. 性能基准建立：通过基础性能测试了解系统瓶颈，为优化提供数据支撑

可选测试场景：

| 测试类型   | 验证点                  | 工具               | 优先级         |
|--------|----------------------|------------------|-------------|
| 延迟测试   | 单次查询的 P50/P95/P99 延迟 | pytest-benchmark | 中（可帮助识别慢查询） |
| 吞吐量测试  | 并发查询时的 QPS 上限        | locust           | 低（本地单用户无需求） |
| 内存泄漏检测 | 长时间运行后的内存占用          | memory_profiler  | 低（短期运行无影响）  |
| 向量库性能  | 不同数据规模下的查询速度         | 自定义 Benchmark    | 中（验证扩展性）    |

4.5 测试工具链与 CI/CD 集成

本地开发工作流：

- 快速验证：仅运行单元测试，秒级反馈
- 完整验证：单元测试 + 集成测试，生成覆盖率报告
- 质量评估：定期执行 RAG 质量测试，监控指标变化

CI/CD Pipeline 设计（可选）：

说明：本地项目不强制要求 CI/CD，但配置自动化测试流程有助于代码质量保障与持续集成实践。

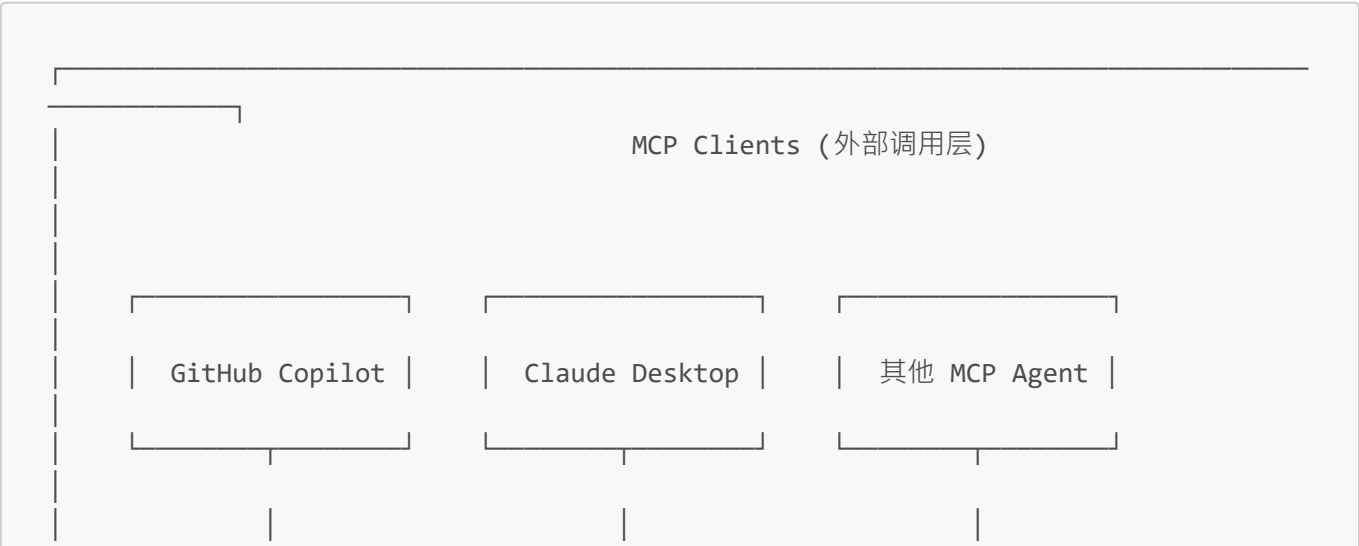
- 单元测试阶段：每次提交自动触发，验证基础功能，生成覆盖率报告
- 集成测试阶段：单元测试通过后执行，验证模块协作
- 质量评估阶段：PR 触发，运行完整的 RAG 质量测试，发布评估报告

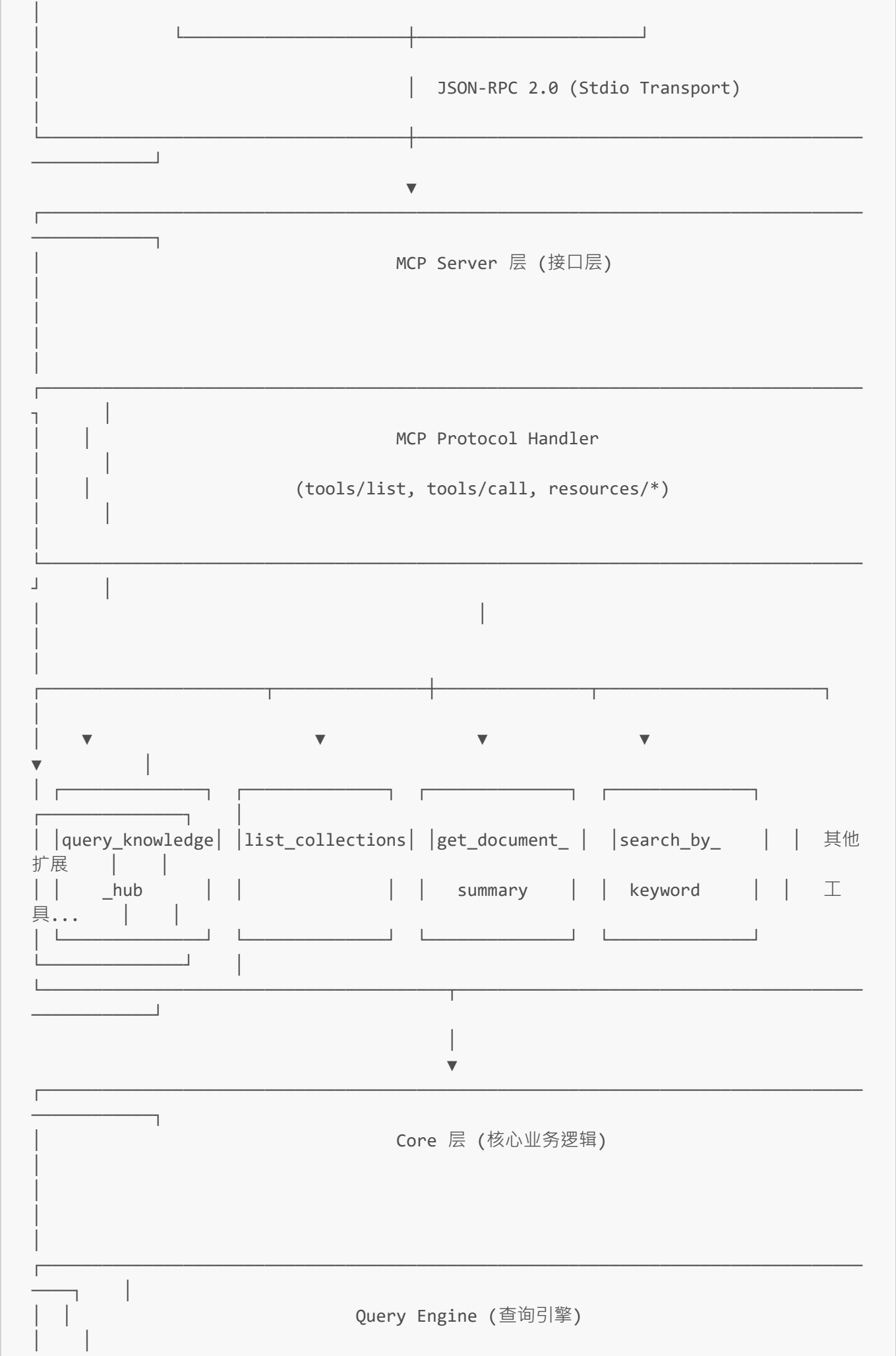
测试覆盖率目标：

- 单元测试：核心逻辑覆盖率 ≥ 80%
- 集成测试：关键路径覆盖率 100%（如 Ingestion、Hybrid Search）
- E2E 测试：核心用户场景覆盖率 100%（至少 3 个关键流程）

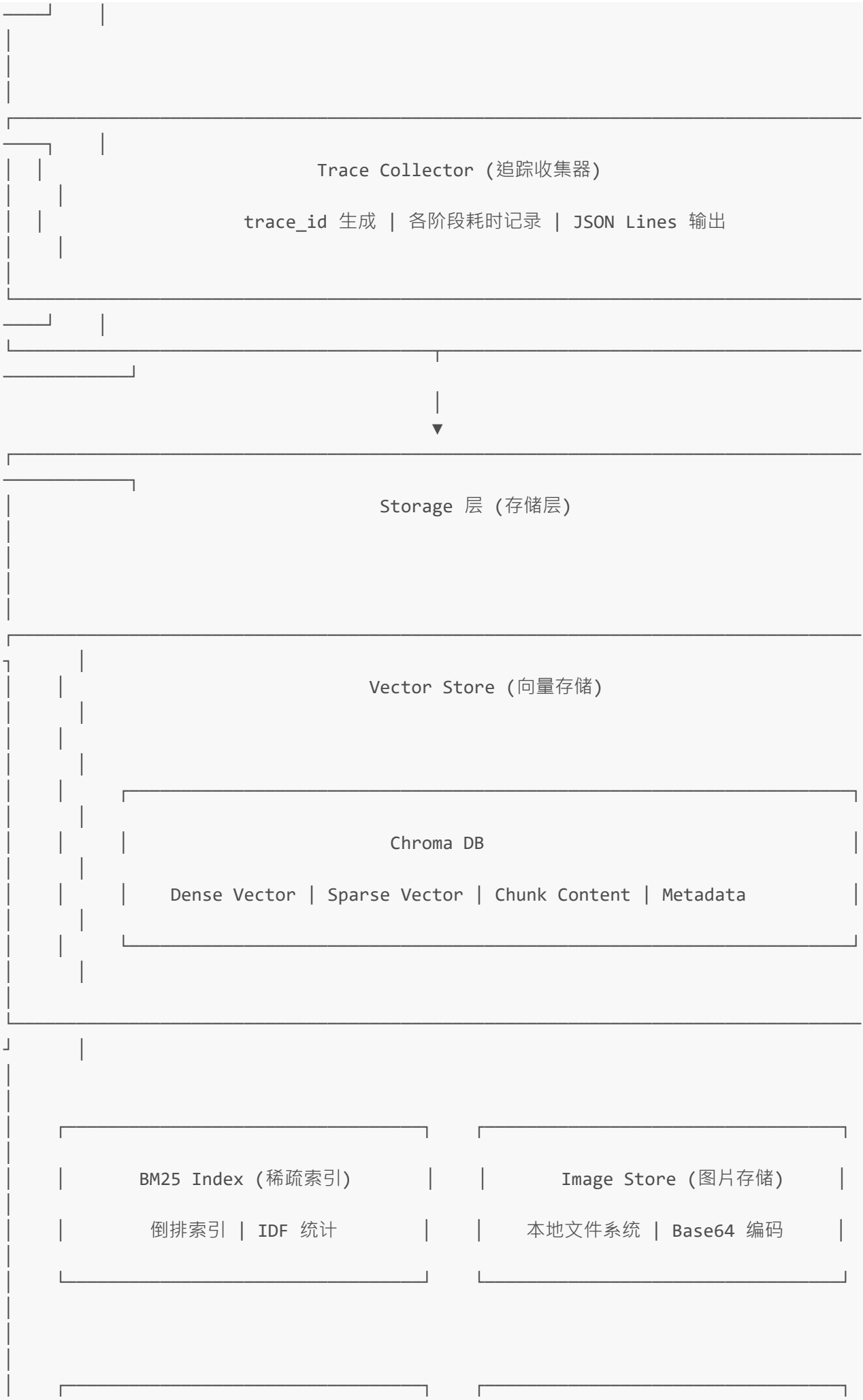
5. 系统架构与模块设计

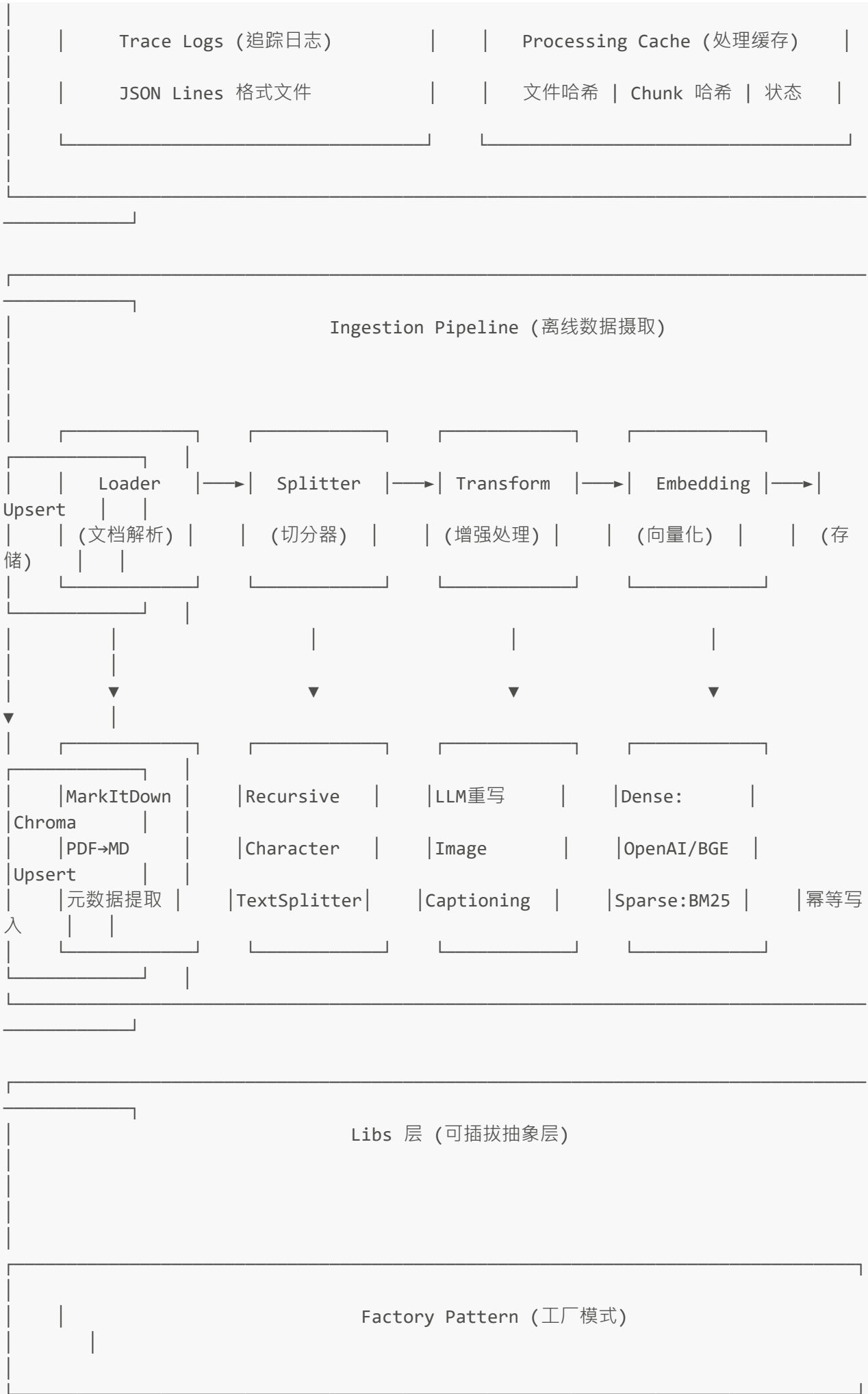
5.1 整体架构图

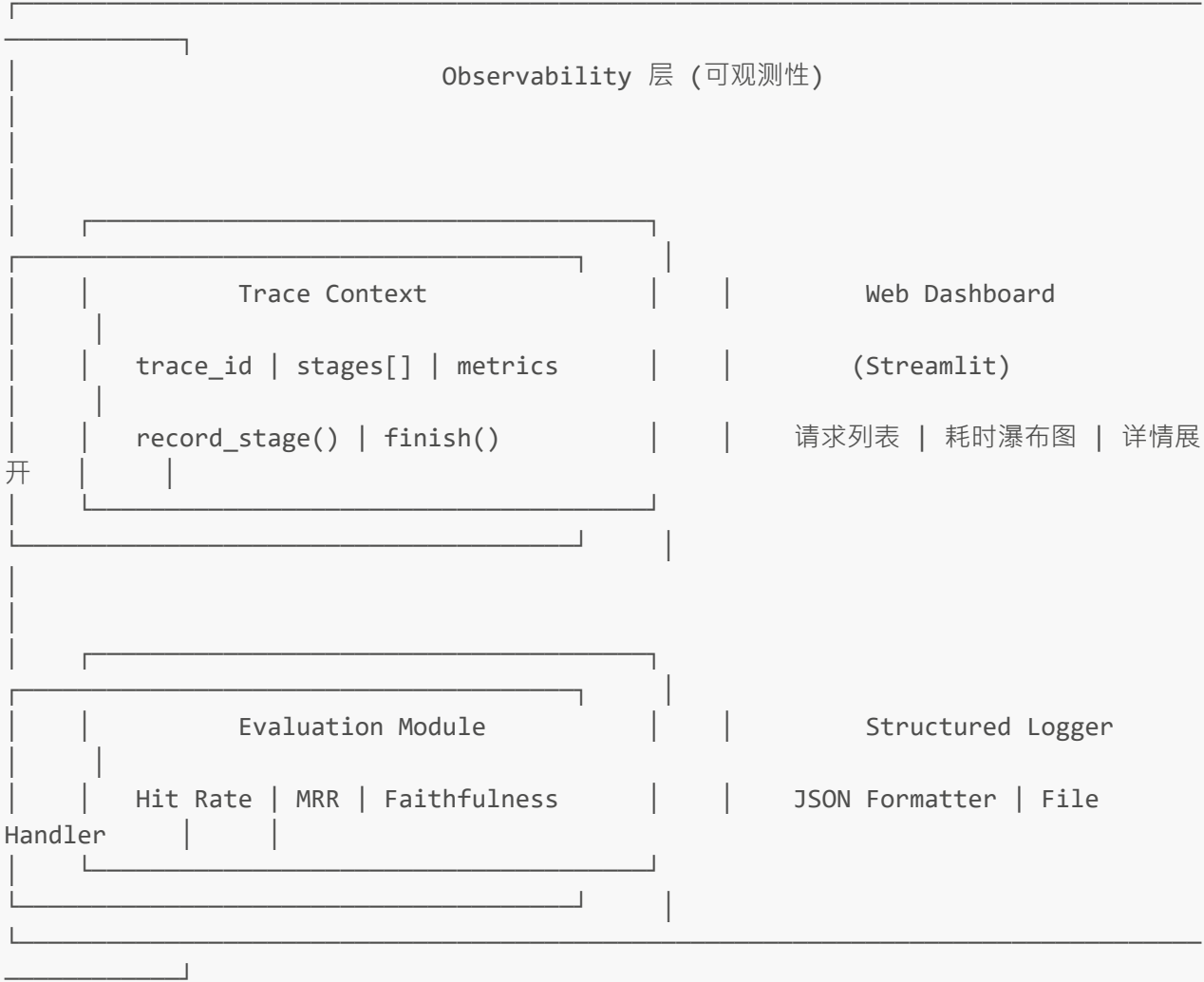
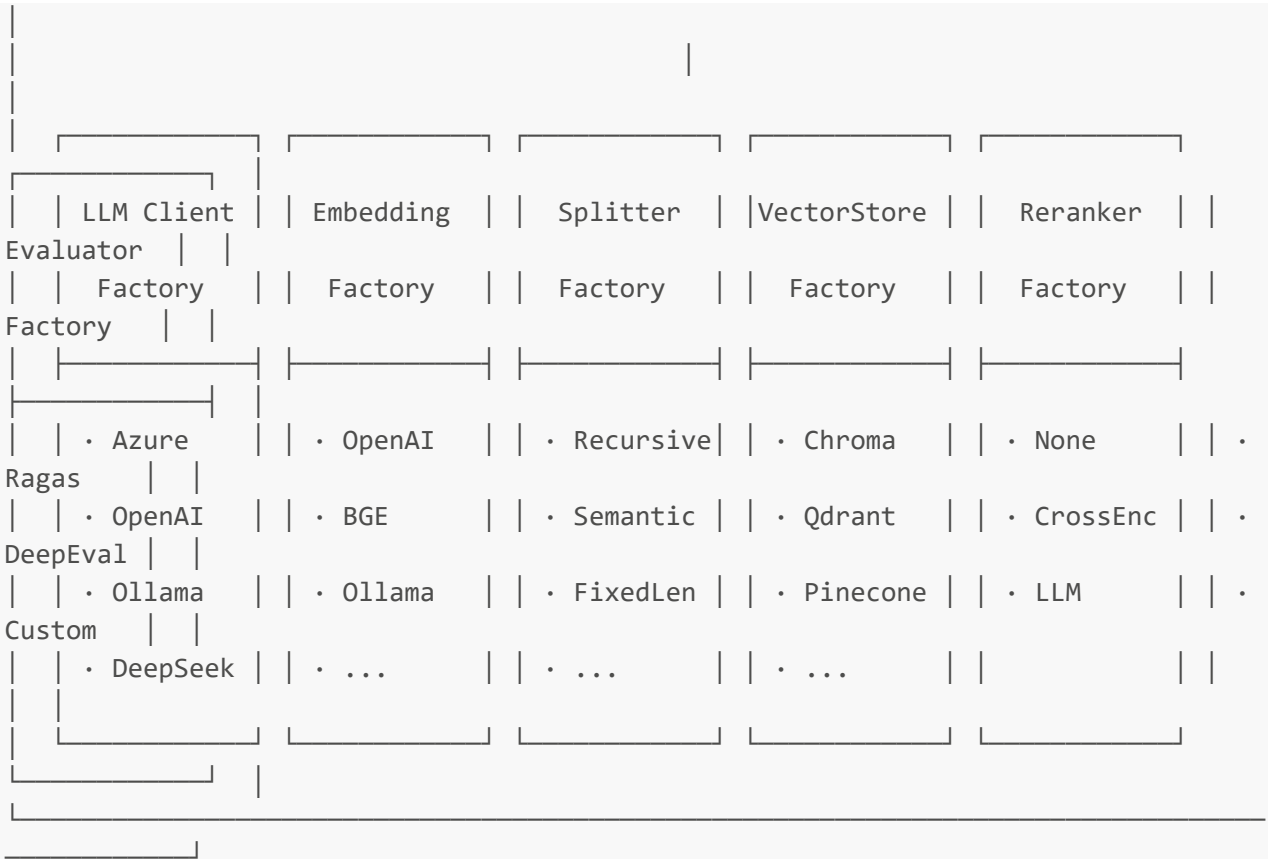












5.2 目录结构

|                                        |                                                       |
|----------------------------------------|-------------------------------------------------------|
| smart-knowledge-hub/                   |                                                       |
| ├─ config/                             | # 配置文件目录                                              |
| │   └─ settings.yaml                   | # 主配置文件 (LLM/Embedding/VectorStore 配置)                |
| ├─ prompts/                            | # Prompt 模板目录                                         |
| │   └─ image_captioning.txt            | # 图片描述生成 Prompt                                       |
| │   └─ chunk_refinement.txt            | # Chunk 重写 Prompt                                     |
| │   └─ rerank.txt                      | # LLM Rerank Prompt                                   |
| ├─ src/                                | # 源代码主目录                                              |
| │   └─ mcp_server/                     | # MCP Server 层 (接口层)                                  |
| │       └─ __init__.py                 |                                                       |
| │       └─ server.py                   | # MCP Server 入口 (Stdio Transport)                     |
| │       └─ protocol_handler.py         | # JSON-RPC 协议处理                                       |
| │       └─ tools/                      | # MCP Tools 定义                                        |
| │           └─ __init__.py             |                                                       |
| │           └─ query_knowledge_hub.py  | # 主检索工具                                               |
| │           └─ list_collections.py     | # 列出集合工具                                              |
| │           └─ get_document_summary.py | # 文档摘要工具                                              |
| │   └─ core/                           | # Core 层 (核心业务逻辑)                                     |
| │       └─ __init__.py                 |                                                       |
| │       └─ settings.py                 | # 配置加载与校验 (Settings: load_settings/validate_settings) |
| ├─ query_engine/                       | # 查询引擎模块                                              |
| │   └─ __init__.py                     |                                                       |
| │   └─ query_processor.py              | # 查询预处理 (关键词提取/查询扩展)                                  |
| │   └─ hybrid_search.py                | # 混合检索引擎 (Dense + Sparse + RRF)                       |
| │   └─ dense_retriever.py              | # 稠密向量检索                                              |
| │   └─ sparse_retriever.py             | # 稀疏检索 (BM25)                                         |
| │   └─ fusion.py                       | # 结果融合 (RRF 算法)                                       |
| │   └─ reranker.py                     | # 重排序模块 (None/CrossEncoder/LLM)                       |
| ├─ response/                           | # 响应构建模块                                              |
| │   └─ __init__.py                     |                                                       |
| │   └─ response_builder.py             | # 响应构建器                                               |
| │   └─ citation_generator.py           | # 引用生成器                                               |
| │   └─ multimodal_assembler.py         | # 多模态内容组装 (Text + Image)                              |
| ├─ trace/                              | # 追踪模块                                                |
| │   └─ __init__.py                     |                                                       |
| │   └─ trace_context.py                | # 追踪上下文 (trace_id/stages)                             |
| │   └─ trace_collector.py              | # 追踪收集器                                               |
| ├─ ingestion/                          | # Ingestion Pipeline (离线数据摄取)                         |
| │   └─ __init__.py                     |                                                       |
| │   └─ pipeline.py                     | # Pipeline 主流程编排                                      |



|                                |                                     |
|--------------------------------|-------------------------------------|
| └─ transform/                  | # Transform 模块 (增强处理)               |
| └─ ── __init__.py              |                                     |
| └─ ── base_transform.py        | # Transform 抽象基类                    |
| └─ ── chunk_refiner.py         | # Chunk 智能重组/去噪                     |
| └─ ── metadata_enricher.py     | # 语义元数据注入 (Title/Summary/Tags)      |
| └─ ── image_captioner.py       | # 图片描述生成 (Vision LLM)               |
| └─ embedding/                  | # Embedding 模块 (向量化)                |
| └─ ── __init__.py              |                                     |
| └─ ── dense_encoder.py         | # 稠密向量编码                            |
| └─ ── sparse_encoder.py        | # 稀疏向量编码 (BM25)                     |
| └─ ── batch_processor.py       | # 批处理优化                             |
| └─ storage/                    | # Storage 模块 (存储)                   |
| └─ ── __init__.py              |                                     |
| └─ ── vector_upserter.py       | # 向量库 Upsert                        |
| └─ ── bm25_indexer.py          | # BM25 索引构建                         |
| └─ ── image_storage.py         | # 图片文件存储                            |
| └─ libs/                       | # Libs 层 (可插拔抽象层)                   |
| └─ ── __init__.py              |                                     |
| └─ loader/                     | # Loader 抽象 (文档加载)                  |
| └─ ── __init__.py              |                                     |
| └─ ── base_loader.py           | # Loader 抽象基类                       |
| └─ ── pdf_loader.py            | # PDF Loader (MarkItDown)           |
| └─ ── file_integrity.py        | # 文件完整性检查 (SHA256 哈希)               |
| └─ llm/                        | # LLM 抽象                            |
| └─ ── __init__.py              |                                     |
| └─ ── base_llm.py              | # LLM 抽象基类                          |
| └─ ── llm_factory.py           | # LLM 工厂                            |
| └─ ── azure_llm.py             | # Azure OpenAI 实现                   |
| └─ ── openai_llm.py            | # OpenAI 实现                         |
| └─ ── ollama_llm.py            | # Ollama 本地模型实现                     |
| └─ ── deepseek_llm.py          | # DeepSeek 实现                       |
| └─ embedding/                  | # Embedding 抽象                      |
| └─ ── __init__.py              |                                     |
| └─ ── base_embedding.py        | # Embedding 抽象基类                    |
| └─ ── embedding_factory.py     | # Embedding 工厂                      |
| └─ ── openai_embedding.py      | # OpenAI Embedding 实现               |
| └─ ── local_embedding.py       | # 本地模型实现 (BGE/Ollama)               |
| └─ splitter/                   | # Splitter 抽象 (切分策略)                |
| └─ ── __init__.py              |                                     |
| └─ ── base_splitter.py         | # Splitter 抽象基类                     |
| └─ ── splitter_factory.py      | # Splitter 工厂                       |
| └─ ── recursive_splitter.py    | # RecursiveCharacterTextSplitter 实现 |
| └─ ── semantic_splitter.py     | # 语义切分实现                            |
| └─ ── fixed_length_splitter.py | # 定长切分实现                            |
| └─ vector_store/               | # VectorStore 抽象                    |

```

├── __init__.py
├── base_vector_store.py      # VectorStore 抽象基类
├── vector_store_factory.py  # VectorStore 工厂
├── chroma_store.py          # Chroma 实现
├── reranker/                # Reranker 抽象
│   ├── __init__.py
│   ├── base_reranker.py     # Reranker 抽象基类
│   ├── reranker_factory.py  # Reranker 工厂
│   ├── cross_encoder_reranker.py # CrossEncoder 实现
│   └── llm_reranker.py      # LLM Rerank 实现
├── evaluator/              # Evaluator 抽象
│   ├── __init__.py
│   ├── base_evaluator.py    # Evaluator 抽象基类
│   ├── evaluator_factory.py # Evaluator 工厂
│   ├── ragas_evaluator.py   # Ragas 实现
│   └── custom_evaluator.py  # 自定义指标实现
├── observability/          # Observability 层 (可观测性)
│   ├── __init__.py
│   ├── logger.py            # 结构化日志 (JSON Formatter)
│   ├── dashboard/          # Web Dashboard
│   │   ├── __init__.py
│   │   └── app.py           # Streamlit Dashboard 应用
│   ├── evaluation/         # 评估模块
│   │   ├── __init__.py
│   │   └── eval_runner.py   # 评估执行器
├── data/                   # 数据目录
│   ├── documents/          # 原始文档存放
│   │   └── {collection}/    # 按集合分类
│   ├── images/             # 提取的图片存放
│   │   └── {collection}/    # 按集合分类
│   ├── db/                 # 数据库文件
│   │   ├── chroma/         # Chroma 向量库
│   │   └── bm25/           # BM25 索引
├── cache/                  # 缓存目录
│   ├── embeddings/         # Embedding 缓存 (按内容哈希)
│   ├── captions/           # 图片描述缓存
│   └── processing/         # 处理状态缓存 (文件哈希/Chunk 哈希)
├── logs/                   # 日志目录
│   ├── traces.jsonl        # 追踪日志 (JSON Lines)
│   └── app.log             # 应用日志
├── tests/                  # 测试目录
│   ├── unit/               # 单元测试
│   │   ├── test_loader/
│   │   ├── test_splitter/
│   │   ├── test_transform/
│   │   ├── test_embedding/
│   │   └── test_retrieval/

```

```
├── test_reranker/
├── integration/                                # 集成测试
│   ├── test_ingestion_pipeline.py
│   ├── test_hybrid_search.py
│   └── test_mcp_server.py
├── e2e/                                        # 端到端测试
│   ├── test_data_ingestion.py
│   ├── test_recall.py
│   └── test_mcp_client.py
├── fixtures/                                # 测试数据
│   ├── sample_documents/
│   └── golden_test_set.json
├── scripts/                                 # 脚本目录
│   ├── ingest.py                            # 数据摄取脚本
│   ├── evaluate.py                          # 评估运行脚本
│   └── start_dashboard.py                   # Dashboard 启动脚本
├── main.py                                  # MCP Server 启动入口
├── pyproject.toml                           # Python 项目配置
├── requirements.txt                         # 依赖列表
└── README.md                               # 项目说明
```

5.3 模块说明

5.3.1 MCP Server 层

| 模块                  | 职责                                   | 关键技术点                            |
|---------------------|--------------------------------------|----------------------------------|
| server.py           | MCP Server 主入口，处理 Stdio Transport 通信 | Python MCP SDK, JSON-RPC 2.0     |
| protocol_handler.py | 协议解析与能力协商                            | initialize、tools/list、tools/call |
| tools/*             | 对外暴露的工具函数实现                          | 装饰器定义，参数校验，响应格式化                 |

5.3.2 Core 层

| 模块                  | 职责         | 关键技术点                                                   |
|---------------------|------------|---------------------------------------------------------|
| settings.py         | 配置加载与校验    | 读取 config/settings.yaml，解析为 Settings，必填字段校验 (fail-fast) |
| query_processor.py  | 查询预处理      | 关键词提取、同义词扩展、Metadata 解析                                 |
| hybrid_search.py    | 混合检索编排     | 并行 Dense/Sparse 召回，结果融合                                 |
| dense_retriever.py  | 语义向量检索     | Query Embedding, Cosine Similarity                      |
| sparse_retriever.py | BM25 关键词检索 | 倒排索引查询，TF-IDF 打分                                        |
| fusion.py           | 结果融合       | RRF 算法，排名倒数加权                                           |

| 模块                  | 职责    | 关键技术点                                |
|---------------------|-------|--------------------------------------|
| reranker.py         | 精排重排  | CrossEncoder / LLM Rerank / Fallback |
| response_builder.py | 响应构建  | Citation 生成，多模态组装                    |
| trace_context.py    | 追踪上下文 | trace_id 生成，阶段记录                     |

5.3.3 Ingestion Pipeline 层

| 模块                    | 职责            | 关键技术点                                    |
|-----------------------|---------------|------------------------------------------|
| pipeline.py           | Pipeline 流程编排 | 串行执行，异常处理，增量更新                           |
| pdf_loader.py         | PDF 文档解析      | MarkItDown, Markdown 标准化输出               |
| file_integrity.py     | 文件去重          | SHA256 哈希，增量检测                           |
| recursive_splitter.py | 文本切分          | LangChain RecursiveCharacterTextSplitter |
| chunk_refiner.py      | Chunk 智能重组    | LLM 二次加工，去噪合并                            |
| metadata_enricher.py  | 元数据增强         | Title/Summary/Tags 自动生成                  |
| image_captioner.py    | 图片描述生成        | Vision LLM (GPT-4o / Qwen-VL)            |
| dense_encoder.py      | 稠密向量编码        | OpenAI Embedding / BGE                   |
| sparse_encoder.py     | 稀疏向量编码        | BM25 编码，IDF 统计                           |
| vector_upserter.py    | 向量存储          | Chroma Upsert，幂等写入                       |

5.3.4 Libs 层 (可插拔抽象)

| 抽象接口            | 当前默认实现                  | 可替换选项                      |
|-----------------|-------------------------|----------------------------|
| LLMClient       | Azure OpenAI            | OpenAI / Ollama / DeepSeek |
| EmbeddingClient | OpenAI text-embedding-3 | BGE / Ollama 本地模型          |
| VectorStore     | Chroma                  | Qdrant / Pinecone / Milvus |
| Reranker        | CrossEncoder            | LLM Rerank / None (关闭)     |
| Evaluator       | Ragas                   | DeepEval / 自定义指标           |

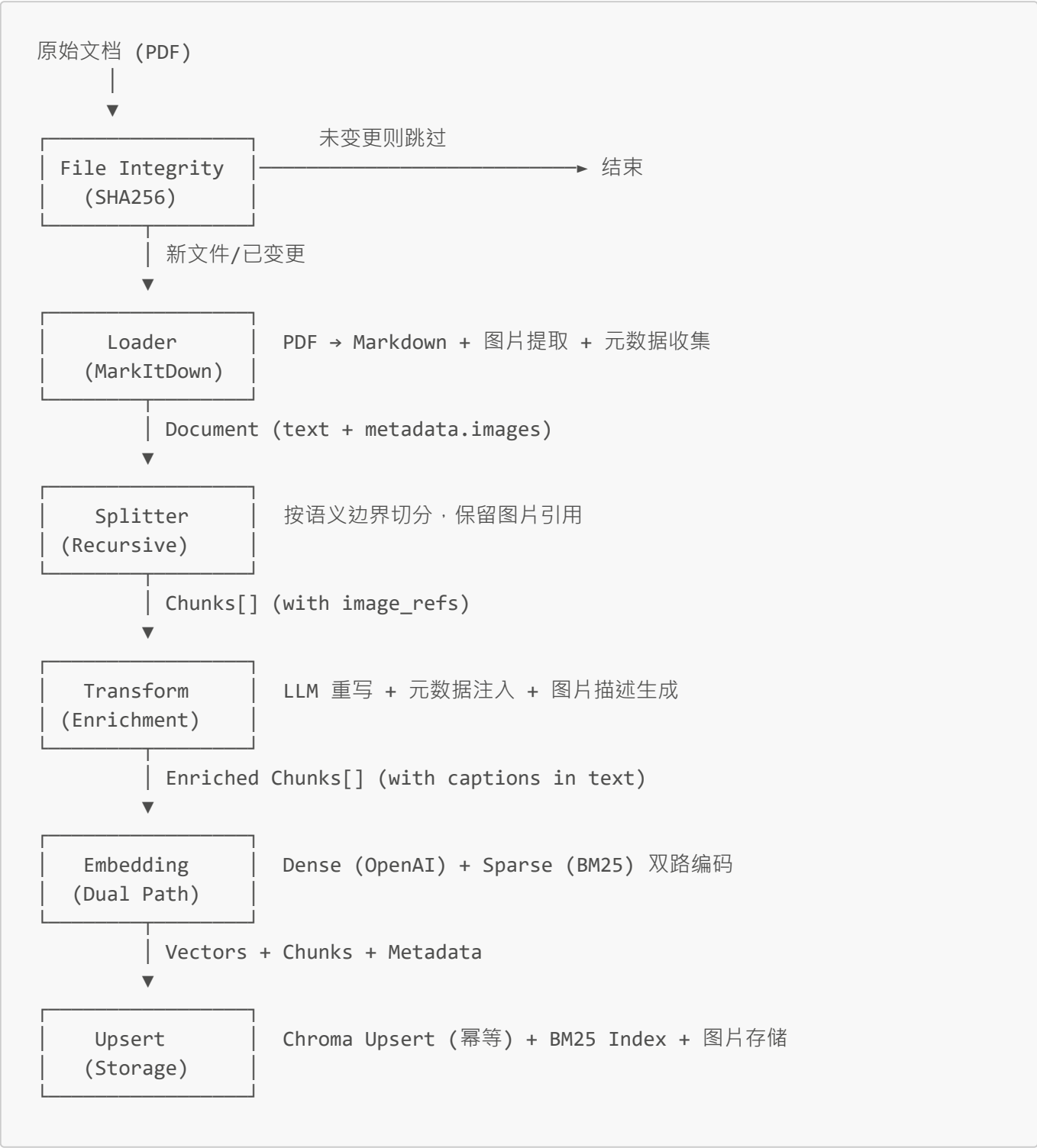
5.3.5 Observability 层

| 模块               | 职责            | 关键技术点                         |
|------------------|---------------|-------------------------------|
| logger.py        | 结构化日志         | JSON Formatter, JSON Lines 输出 |
| trace_context.py | 请求级追踪         | trace_id，阶段耗时记录               |
| dashboard/app.py | Web Dashboard | Streamlit，请求列表，耗时瀑布图          |

| 模块             | 职责   | 关键技术点           |
|----------------|------|-----------------|
| eval_runner.py | 评估执行 | 黄金测试集，指标计算，报告生成 |

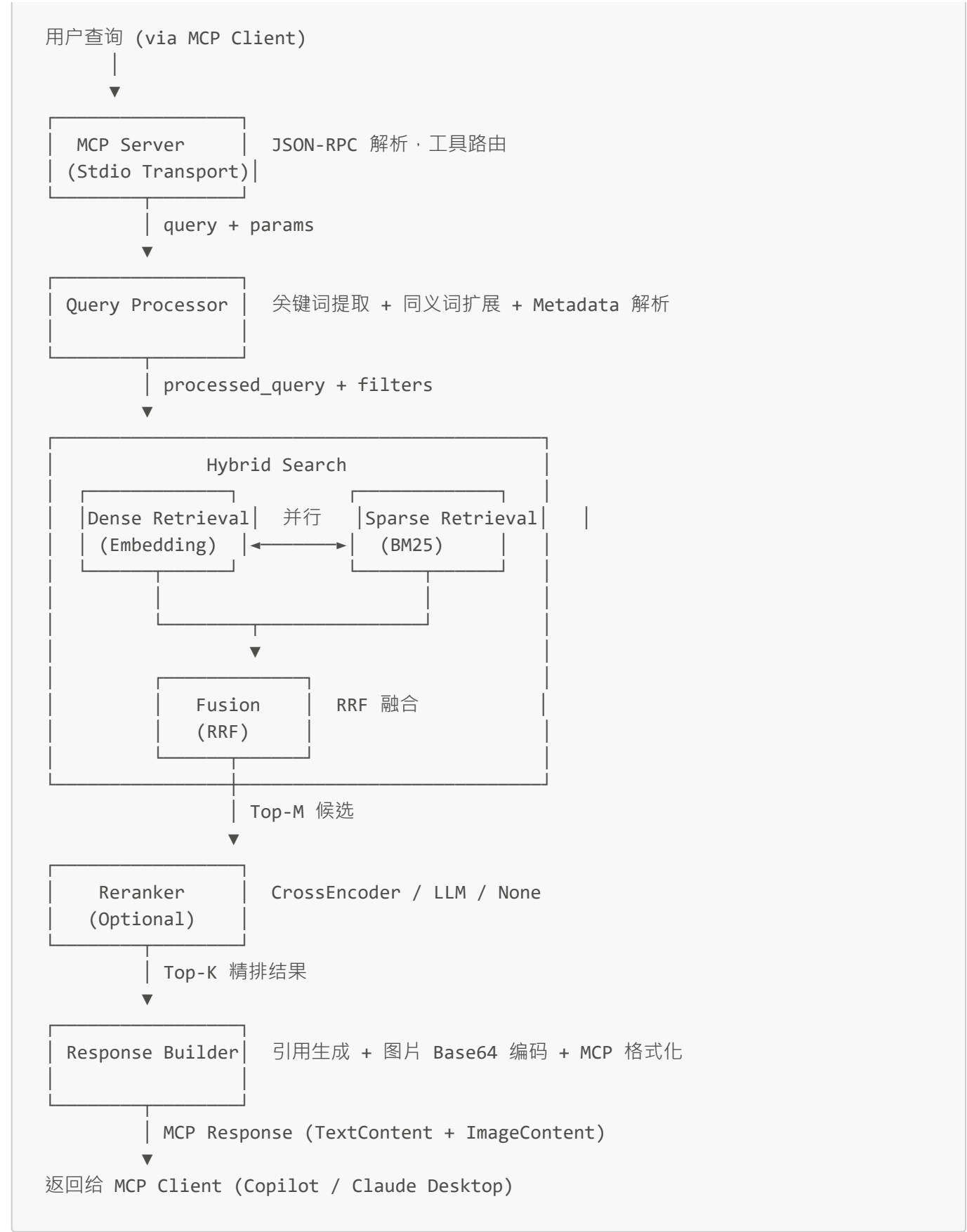
5.4 数据流说明

5.4.1 离线数据摄取流 (Ingestion Flow)



5.4.2 在线查询流 (Query Flow)





5.5 配置驱动设计

系统通过 `config/settings.yaml` 统一配置各组件实现, 支持零代码切换:

```
# config/settings.yaml 示例
```

```
# LLM 配置
llm:
  provider: azure          # azure | openai | ollama | deepseek
  model: gpt-4o
  azure_endpoint: "..."
  api_key: "${AZURE_API_KEY}"

# Embedding 配置
embedding:
  provider: openai         # openai | ollama | local
  model: text-embedding-3-small

# Vision LLM 配置 (图片描述)
vision_llm:
  provider: azure          # azure | dashscope (Qwen-VL)
  model: gpt-4o

# 向量存储配置
vector_store:
  backend: chroma          # chroma | qdrant | pinecone
  persist_path: ./data/db/chroma

# 检索配置
retrieval:
  sparse_backend: bm25      # bm25 | elasticsearch
  fusion_algorithm: rrf     # rrf | weighted_sum
  top_k_dense: 20
  top_k_sparse: 20
  top_k_final: 10

# 重排配置
rerank:
  backend: cross_encoder   # none | cross_encoder | llm
  model: cross-encoder/ms-marco-MiniLM-L-6-v2
  top_m: 30

# 评估配置
evaluation:
  backends: [ragas, custom]
  golden_test_set: ./tests/fixtures/golden_test_set.json

# 可观测性配置
observability:
  enabled: true
  log_file: ./logs/traces.jsonl
  dashboard_port: 8501
```

## 5.6 扩展性设计要点

1. **新增 LLM Provider**：实现 `BaseLLM` 接口，在 `llm_factory.py` 注册，配置文件指定 `provider` 即可
2. **新增文档格式**：实现 `BaseLoader` 接口，在 `Pipeline` 中注册对应文件扩展名的处理器
3. **新增检索策略**：实现检索接口，在 `hybrid_search.py` 中组合调用

4. 新增评估指标：实现 `BaseEvaluator` 接口，在配置中添加到 `backends` 列表

6. 项目排期

排期原则（严格对齐本 DEV\_SPEC 的架构分层与目录结构）

- 只按本文档设计落地：以第 5.2 节目录树为“交付清单”，每一步都要在文件系统上产生可见变化。
- 1 小时一个可验收增量：每个小阶段（≈1h）都必须同时给出“验收标准 + 测试方法”，尽量做到 TDD。
- 先打通主闭环，再补齐默认实现：优先做“可跑通的端到端路径（Ingestion → Retrieval → MCP Tool）”，并在 Libs 层补齐可运行的默认后端实现，避免出现“只有接口没有实现”的空转。
- 外部依赖可替换/可 Mock：LLM/Embedding/Vision/VectorStore 的真实调用在单元测试中一律用 Fake/Mock，集成测试再开真实后端（可选）。

阶段总览（大阶段 → 目的）

1. 阶段 A：工程骨架与测试基座
  - 目的：建立可运行、可配置、可测试的工程骨架；后续所有模块都能以 TDD 方式落地。
2. 阶段 B：Libs 可插拔层（Factory + Base 接口 + 默认可运行实现）
  - 目的：把“可替换”变成代码事实；并补齐可运行的默认后端实现，确保 Core / Ingestion 不仅“可编译”，还可在真实环境跑通。
3. 阶段 C：Ingestion Pipeline（PDF→MD→Chunk→Embedding→Upsert）
  - 目的：离线摄取链路跑通，能把样例文档写入向量库/BM25 索引并支持增量。
4. 阶段 D：Retrieval（Dense + Sparse + RRF + 可选 Rerank）
  - 目的：在线查询链路跑通，得到 Top-K chunks（含引用信息），并具备稳定回退策略。
5. 阶段 E：MCP Server 层与 Tools 落地
  - 目的：按 MCP 标准暴露 tools，让 Copilot/Claude 可直接调用查询能力。
6. 阶段 F：Observability + Evaluation 闭环
  - 目的：把“可调试、可量化”落地：trace.jsonl、Dashboard、golden set 回归。
7. 阶段 G：端到端验收与文档收口
  - 目的：补齐 E2E 与运行脚本；确保“开箱即用 + 可复现实验”。

📊 进度跟踪表 (Progress Tracking)

状态说明：[ ] 未开始 | [~] 进行中 | [x] 已完成

更新时间：每完成一个子任务后更新对应状态

阶段 A：工程骨架与测试基座

| 任务编号 | 任务名称                | 状态  | 完成日期       | 备注 |
|------|---------------------|-----|------------|----|
| A1   | 初始化目录树与最小可运行入口      | [x] | 2026-01-26 |    |
| A2   | 引入 pytest 并建立测试目录约定 | [x] | 2026-01-26 |    |



| 任务编号 | 任务名称                | 状态  | 完成日期       | 备注 |
|------|---------------------|-----|------------|----|
| A3   | 配置加载与校验 ( Settings) | [x] | 2026-01-26 |    |

阶段 B：Libs 可插拔层

| 任务编号 | 任务名称                           | 状态  | 完成日期 | 备注 |
|------|--------------------------------|-----|------|----|
| B1   | LLM 抽象接口与工厂                    | [ ] | -    |    |
| B2   | Embedding 抽象接口与工厂              | [ ] | -    |    |
| B3   | Splitter 抽象接口与工厂               | [ ] | -    |    |
| B4   | VectorStore 抽象接口与工厂            | [ ] | -    |    |
| B5   | Reranker 抽象接口与工厂 ( 含 None 回退 ) | [ ] | -    |    |
| B6   | Evaluator 抽象接口与工厂              | [ ] | -    |    |
| B7.1 | OpenAI-Compatible LLM 实现       | [ ] | -    |    |
| B7.2 | Ollama LLM 实现                  | [ ] | -    |    |
| B7.3 | OpenAI Embedding 实现            | [ ] | -    |    |
| B7.4 | Local Embedding 实现             | [ ] | -    |    |
| B7.5 | Recursive Splitter 默认实现        | [ ] | -    |    |
| B7.6 | ChromaStore 默认实现               | [ ] | -    |    |
| B7.7 | LLM Reranker 实现                | [ ] | -    |    |
| B7.8 | Cross-Encoder Reranker 实现      | [ ] | -    |    |

阶段 C：Ingestion Pipeline MVP

| 任务编号 | 任务名称                              | 状态  | 完成日期 | 备注 |
|------|-----------------------------------|-----|------|----|
| C1   | 定义核心数据模型 ( Document/Chunk/Record) | [ ] | -    |    |
| C2   | 文件完整性检查 ( SHA256)                 | [ ] | -    |    |
| C3   | Loader 抽象基类与 PDF Loader           | [ ] | -    |    |
| C4   | Splitter 集成 ( 调用 Libs)            | [ ] | -    |    |
| C5   | Transform 基类 + ChunkRefiner       | [ ] | -    |    |
| C6   | MetadataEnricher                  | [ ] | -    |    |
| C7   | ImageCaptioner                    | [ ] | -    |    |
| C8   | DenseEncoder                      | [ ] | -    |    |
| C9   | SparseEncoder                     | [ ] | -    |    |

| 任务编号 | 任务名称                    | 状态  | 完成日期 | 备注 |
|------|-------------------------|-----|------|----|
| C10  | BatchProcessor          | [ ] | -    |    |
| C11  | VectorUpserter          | [ ] | -    |    |
| C12  | BM25Indexer             | [ ] | -    |    |
| C13  | ImageStorage            | [ ] | -    |    |
| C14  | Pipeline 编排 ( MVP 串起来 ) | [ ] | -    |    |
| C15  | 脚本入口 ingest.py          | [ ] | -    |    |

阶段 D：Retrieval MVP

| 任务编号 | 任务名称                              | 状态  | 完成日期 | 备注 |
|------|-----------------------------------|-----|------|----|
| D1   | QueryProcessor ( 关键词提取 + filters) | [ ] | -    |    |
| D2   | DenseRetriever                    | [ ] | -    |    |
| D3   | SparseRetriever (BM25)            | [ ] | -    |    |
| D4   | RRF Fusion                        | [ ] | -    |    |
| D5   | MetadataFilter                    | [ ] | -    |    |
| D6   | Rerank 集成与 Fallback               | [ ] | -    |    |
| D7   | RetrievalPipeline 编排              | [ ] | -    |    |

阶段 E：MCP Server 层与 Tools

| 任务编号 | 任务名称                             | 状态  | 完成日期 | 备注 |
|------|----------------------------------|-----|------|----|
| E1   | MCP Server 骨架 ( Stdio Transport) | [ ] | -    |    |
| E2   | query_knowledge_hub Tool         | [ ] | -    |    |
| E3   | list_collections Tool            | [ ] | -    |    |
| E4   | get_document_summary Tool        | [ ] | -    |    |
| E5   | 多模态返回 ( ImageContent)            | [ ] | -    |    |
| E6   | 错误处理与协议合规                        | [ ] | -    |    |

阶段 F：Observability + Evaluation

| 任务编号 | 任务名称                | 状态  | 完成日期 | 备注 |
|------|---------------------|-----|------|----|
| F1   | TraceContext 与结构化日志 | [ ] | -    |    |
| F2   | 各阶段 Trace 集成        | [ ] | -    |    |

| 任务编号 | 任务名称                      | 状态  | 完成日期 | 备注 |
|------|---------------------------|-----|------|----|
| F3   | Streamlit Dashboard       | [ ] | -    |    |
| F4   | Golden Test Set 与回归测试     | [ ] | -    |    |
| F5   | Ragas/Custom Evaluator 集成 | [ ] | -    |    |

阶段 G：端到端验收与文档收口

| 任务编号 | 任务名称                       | 状态  | 完成日期 | 备注 |
|------|----------------------------|-----|------|----|
| G1   | E2E 测试用例补齐                 | [ ] | -    |    |
| G2   | 运行脚本与 README 完善            | [ ] | -    |    |
| G3   | MCP 配置示例 ( Copilot/Claude) | [ ] | -    |    |
| G4   | 最终验收与文档检查                  | [ ] | -    |    |

 总体进度

| 阶段   | 总任务数 | 已完成 | 进度 |
|------|------|-----|----|
| 阶段 A | 3    | 0   | 0% |
| 阶段 B | 14   | 0   | 0% |
| 阶段 C | 15   | 0   | 0% |
| 阶段 D | 7    | 0   | 0% |
| 阶段 E | 6    | 0   | 0% |
| 阶段 F | 5    | 0   | 0% |
| 阶段 G | 4    | 0   | 0% |
| 总计   | 54   | 0   | 0% |

阶段 A：工程骨架与测试基座（目标：先可导入，再可测试）

A1：初始化目录树与最小可运行入口 

- 目标：在 repo 根目录创建第 5.2 节所述目录骨架与空模块文件（可 import）。
- 修改文件：
  - main.py
  - pyproject.toml
  - README.md
  - src/\*\*/\_\_init\_\_.py（按目录树补齐）
  - config/settings.yaml（最小可解析配置）
  - config/prompts/image\_captioning.txt（可先放占位内容，后续阶段补充 Prompt）
  - config/prompts/chunk\_refinement.txt（可先放占位内容，后续阶段补充 Prompt）

- `config/prompts/rerank.txt` (可先放占位内容，后续阶段补充 Prompt)
- 实现类/函数：无 (仅骨架)。
- 实现类/函数：无 (仅骨架，不实现业务逻辑)。
- 实现类/函数：为当前项目创建一个虚拟环境模块。
- 验收标准：
- 目录结构与 DEV\_SPEC 5.2 一致 (至少把对应目录创建出来)。
- `config/prompts/` 目录存在，且三个 `prompt` 文件可被读取 (即使只是占位文本)。
- 能导入关键顶层包 (与目录结构一一对应)：
  - `python -c "import mcp_server; import core; import ingestion; import libs; import observability"`
- 可以启动虚拟环境模块
- 测试方法：运行 `python -m compileall src` (仅做语法/可导入性检查；pytest 基座在 A2 建立)。

## A2：引入 pytest 并建立测试目录约定

- 目标：建立 `tests/unit|integration|e2e|fixtures` 目录与 pytest 运行基座。
- 修改文件：
  - `pyproject.toml` (添加 pytest 配置：testpaths、markers 等)
  - `tests/unit/test_smoke_imports.py`
  - `tests/fixtures/sample_documents/` (放 1 个最小样例文档占位)
- 实现类/函数：无。
- 实现类/函数：无 (新增的是测试文件与 pytest 配置)。
- 验收标准：
  - `pytest -q` 可运行并通过。
  - 至少 1 个冒烟测试 (例如 `tests/unit/test_smoke_imports.py` 只做关键包 import 校验)。
- 测试方法：`pytest -q tests/unit/test_smoke_imports.py`。

## A3：配置加载与校验 (Settings)

- 目标：实现读取 `config/settings.yaml` 的配置加载器，并在启动时校验关键字段存在。
- 修改文件：
  - `main.py` (启动时调用 `load_settings()`，缺字段直接 fail-fast 退出)
  - `src/observability/logger.py` (先占位：提供 `get_logger`，stderr 输出)
  - `src/core/settings.py` (新增：集中放 Settings 数据结构与加载/校验逻辑)
  - `config/settings.yaml` (补齐字段：  
llm/embedding/vector\_store/retrieval/rerank/evaluation/observability)
  - `tests/unit/test_config_loading.py`
- 实现类/函数：
  - `Settings` (dataclass：只做结构与最小校验；不在这里做任何网络/IO 的“业务初始化”)
  - `load_settings(path: str) -> Settings` (读取 YAML -> 解析为 Settings -> 校验必填字段)
  - `validate_settings(settings: Settings) -> None` (把“必填字段检查”集中化，错误信息包含字段路径，例如 `embedding.provider`)
- 验收标准：
  - `main.py` 启动时能成功加载 `config/settings.yaml` 并拿到 `Settings` 对象。
  - 删除/缺失关键字段时 (例如 `embedding.provider`)，启动或 `load_settings()` 抛出“可读错误” (明确指出缺的是哪个字段)。
- 测试方法：`pytest -q tests/unit/test_config_loading.py`。

## 阶段 B：Libs 可插拔层（目标：Factory 可工作，且至少有“默认后端”可跑通端到端）

### B1：LLM 抽象接口与工厂

- 目标：定义 `BaseLLM` 与 `LLMFactory`，支持按配置选择 provider。
- 修改文件：
  - `src/libs/llm/base_llm.py`
  - `src/libs/llm/llm_factory.py`
  - `tests/unit/test_llm_factory.py`
- 实现类/函数：
  - `BaseLLM.chat(messages) -> str`（或统一 response 对象）
  - `LLMFactory.create(settings) -> BaseLLM`
- 验收标准：在测试里用 Fake provider（测试内 stub）验证工厂路由逻辑。
- 测试方法：`pytest -q tests/unit/test_llm_factory.py`。

### B2：Embedding 抽象接口与工厂

- 目标：定义 `BaseEmbedding` 与 `EmbeddingFactory`，支持批量 embed。
- 修改文件：
  - `src/libs/embedding/base_embedding.py`
  - `src/libs/embedding/embedding_factory.py`
  - `tests/unit/test_embedding_factory.py`
- 实现类/函数：
  - `BaseEmbedding.embed(texts: list[str], trace: TraceContext | None = None) -> list[list[float]]`
  - `EmbeddingFactory.create(settings) -> BaseEmbedding`
- 验收标准：Fake embedding 返回稳定向量，工厂按 provider 分流。
- 测试方法：`pytest -q tests/unit/test_embedding_factory.py`。

### B3：Splitter 抽象接口与工厂

- 目标：定义 `BaseSplitter` 与 `SplitterFactory`，支持不同切分策略（Recursive/Semantic/Fixed）。
- 修改文件：
  - `src/libs/splitter/base_splitter.py`
  - `src/libs/splitter/splitter_factory.py`
  - `tests/unit/test_splitter_factory.py`
- 实现类/函数：
  - `BaseSplitter.split_text(text: str, trace: TraceContext | None = None) -> List[str]`
  - `SplitterFactory.create(settings) -> BaseSplitter`
- 验收标准：Factory 能根据配置返回不同类型的 Splitter 实例（测试中可用 Fake 实现）。
- 测试方法：`pytest -q tests/unit/test_splitter_factory.py`。

### B4：VectorStore 抽象接口与工厂（先定义契约）

- 目标：定义 `BaseVectorStore` 与 `VectorStoreFactory`，先不接真实 DB。

- 修改文件：
  - `src/libs/vector_store/base_vector_store.py`
  - `src/libs/vector_store/vector_store_factory.py`
  - `tests/unit/test_vector_store_contract.py`
- 实现类/函数：
  - `BaseVectorStore.upsert(records, trace: TraceContext | None = None)`
  - `BaseVectorStore.query(vector, top_k, filters, trace: TraceContext | None = None)`
- 验收标准：契约测试 ( contract test ) 约束输入输出 shape。
- 测试方法：`pytest -q tests/unit/test_vector_store_contract.py`。

## B5：Reranker 抽象接口与工厂 ( 含 None 回退 )

- 目标：实现 `BaseReranker`、`RerankerFactory`，提供 `NoneReranker` 作为默认回退。
- 修改文件：
  - `src/libs/reranker/base_reranker.py`
  - `src/libs/reranker/reranker_factory.py`
  - `tests/unit/test_reranker_factory.py`
- 实现类/函数：
  - `BaseReranker.rerank(query, candidates, trace: TraceContext | None = None) -> ranked_candidates`
  - `NoneReranker` ( 保持原顺序 )
- 验收标准：backend=None 时不会改变排序；未知 backend 明确报错。
- 测试方法：`pytest -q tests/unit/test_reranker_factory.py`。

## B6：Evaluator 抽象接口与工厂 ( 先做自定义轻量指标 )

- 目标：定义 `BaseEvaluator`、`EvaluatorFactory`，实现最小 `CustomEvaluator` ( 例如 hit\_rate/mrr )。
- 修改文件：
  - `src/libs/evaluator/base_evaluator.py`
  - `src/libs/evaluator/evaluator_factory.py`
  - `src/libs/evaluator/custom_evaluator.py`
  - `tests/unit/test_custom_evaluator.py`
- 验收标准：输入 query + retrieved\_ids + golden\_ids 能输出稳定 metrics。
- 测试方法：`pytest -q tests/unit/test_custom_evaluator.py`。

## B7：补齐 Libs 默认实现 ( 拆分为≈1h可验收增量 )

说明：B7 只补齐与端到端主链路强相关的默认实现 ( LLM/Embedding/Splitter/VectorStore/Reranker )。其余可选扩展 ( 例如额外 splitter 策略、更多 vector store 后端、更多 evaluator 后端等 ) 保持原排期不提前。

### B7.1：OpenAI-Compatible LLM (OpenAI/Azure/DeepSeek)

- 目标：补齐 OpenAI-compatible 的 LLM 实现，确保通过 `LLMFactory` 可创建并可被 mock 测试。
- 修改文件：
  - `src/libs/llm/openai_llm.py`
  - `src/libs/llm/azure_llm.py`

- `src/libs/llm/deepseek_llm.py`
  - `tests/unit/test_llm_providers_smoke.py` (mock HTTP, 不走真实网络)
- 验收标准：
  - 配置不同 `provider` 时工厂路由正确。
  - `chat(messages)` 对输入 `shape` 校验清晰, 异常信息可读 (包含 `provider` 与错误类型)。
- 测试方法: `pytest -q tests/unit/test_llm_providers_smoke.py`。

## B7.2 : Ollama LLM ( 本地后端 )

- 目标: 补齐 `ollama_llm.py`, 支持本地 HTTP endpoint ( 默认 `base_url + model` ), 并可被 mock 测试。
- 修改文件：
  - `src/libs/llm/ollama_llm.py`
  - `tests/unit/test_ollama_llm.py` (mock HTTP)
- 验收标准：
  - `provider=ollama` 时可由 `LLMFactory` 创建。
  - 在连接失败/超时等场景下, 抛出可读错误且不泄露敏感配置。
- 测试方法: `pytest -q tests/unit/test_ollama_llm.py`。

## B7.3 : OpenAI Embedding 实现

- 目标: 补齐 `openai_embedding.py`, 支持批量 `embed(texts)`, 并可被 mock 测试。
- 修改文件：
  - `src/libs/embedding/openai_embedding.py`
  - `tests/unit/test_embedding_providers_smoke.py` (mock HTTP)
- 验收标准：
  - `provider=openai` 时 `EmbeddingFactory` 可创建。
  - 空输入、超长输入有明确行为 ( 报错或截断策略由配置决定 )。
- 测试方法: `pytest -q tests/unit/test_embedding_providers_smoke.py`。

## B7.4 : Local Embedding 实现 ( BGE/Ollama 占位 )

- 目标: 补齐 `local_embedding.py` 的默认实现路径 ( 可先做占位/适配层 ), 并在测试中用 Fake 向量保证链路可跑。
- 修改文件：
  - `src/libs/embedding/local_embedding.py`
  - `tests/unit/test_local_embedding.py`
- 验收标准：
  - `provider=local` 时 `EmbeddingFactory` 可创建。
  - 输出向量维度稳定 ( 配置化或固定假维度 ), 满足 `ingestion/retrieval` 的接口契约。
- 测试方法: `pytest -q tests/unit/test_local_embedding.py`。

## B7.5 : Recursive Splitter 默认实现

- 目标: 补齐 `recursive_splitter.py`, 封装 `LangChain` 的切分逻辑, 作为默认切分器。
- 修改文件：
  - `src/libs/splitter/recursive_splitter.py`
  - `tests/unit/test_recursive_splitter_lib.py`

- 验收标准：
  - `provider=recursive` 时 `SplitterFactory` 可创建。
  - `split_text` 能正确处理 Markdown 结构（标题/代码块不被打断）。
- 测试方法：`pytest -q tests/unit/test_recursive_splitter_lib.py`。

## B7.6：ChromaStore（VectorStore 默认后端）

- 目标：补齐 `chroma_store.py`，支持最小 `upsert(records)` 与 `query(vector, top_k, filters)`，并支持本地持久化目录（例如 `data/db/chroma/`）。
- 修改文件：
  - `src/libs/vector_store/chroma_store.py`
  - `tests/integration/test_chroma_store_roundtrip.py`（可选：标记为 `integration`，允许跳过）
- 验收标准：
  - `provider=chroma` 时 `VectorStoreFactory` 可创建。
  - 在可用环境下完成一次最小 `roundtrip`：`upsert`→`query` 返回 `deterministic` 结果。
- 测试方法：`pytest -q tests/integration/test_chroma_store_roundtrip.py`（可选）。

## B7.7：LLM Reranker（读取 rerank prompt）

- 目标：补齐 `llm_reranker.py`，读取 `config/prompts/rerank.txt` 构造 prompt（测试中可注入替代文本），并可在失败时返回可回退信号。
- 修改文件：
  - `src/libs/reranker/llm_reranker.py`
  - `tests/unit/test_llm_reranker.py`（mock LLM）
- 验收标准：
  - `backend=llm` 时 `RerankerFactory` 可创建。
  - 输出严格结构化（例如 `ranked ids`），不满足 `schema` 时抛出可读错误。
- 测试方法：`pytest -q tests/unit/test_llm_reranker.py`。

## B7.8：Cross-Encoder Reranker（本地/托管模型，占位可跑）

- 目标：补齐 `cross_encoder_reranker.py`，支持对 Top-M candidates 打分排序；测试中用 `mock scorer` 保证 `deterministic`。
- 修改文件：
  - `src/libs/reranker/cross_encoder_reranker.py`
  - `tests/unit/test_cross_encoder_reranker.py`（mock scorer）
- 验收标准：
  - `backend=cross_encoder` 时 `RerankerFactory` 可创建。
  - 提供超时/失败回退信号（供 Core 层 D6 fallback 使用）。
- 测试方法：`pytest -q tests/unit/test_cross_encoder_reranker.py`。

---

## 阶段 C：Ingestion Pipeline MVP（目标：能把 PDF 样例摄取到本地存储）

注：本阶段严格按 5.4.1 的离线数据流落地，并优先实现“增量跳过（SHA256）”。

## C1：定义核心数据模型（Document/Chunk/Record）



- 目标：定义 ingestion 与 retrieval 共用的数据结构（最少字段：text、metadata、ids）。
- 修改文件：
  - `src/ingestion/__init__.py`（若需新增 `src/ingestion/models.py`，需同步在 5.2 目录结构中补充）
  - `tests/unit/test_models.py`
- 实现类/函数（建议）：
  - `Document(id, text, metadata)`
  - `Chunk(id, text, metadata, start_offset, end_offset)`
- 验收标准：模型可序列化（dict/json）并在测试中断言字段稳定。
- 测试方法：`pytest -q tests/unit/test_models.py`。

## C2：文件完整性检查（SHA256）

- 目标：在 Libs 中实现 `file_integrity.py`：计算文件 hash，并提供“是否跳过”的判定接口（先用本地 cache 文件/SQLite 任一实现，后续可替换）。
- 修改文件：
  - `src/libs/loader/file_integrity.py`
  - `tests/unit/test_file_integrity.py`
- 实现类/函数：
  - `compute_sha256(path) -> str`
  - `should_skip(file_hash) -> bool`
  - `mark_success(file_hash)`
- 验收标准：同一文件多次 hash 一致；标记 success 后应 skip。
- 测试方法：`pytest -q tests/unit/test_file_integrity.py`。

## C3：Loader 抽象基类与 PDF Loader 壳子

- 目标：在 Libs 中定义 `BaseLoader`，并实现 `PdfLoader` 的最小行为。
- 修改文件：
  - `src/libs/loader/base_loader.py`
  - `src/libs/loader/pdf_loader.py`
  - `tests/unit/test_loader_pdf_contract.py`
- 实现类/函数：
  - `BaseLoader.load(path) -> Document`
  - `PdfLoader.load(path)`
- 验收标准：对 sample PDF（fixtures）能产出 Document，metadata 至少含 `source_path`。
- 测试方法：`pytest -q tests/unit/test_loader_pdf_contract.py`。

## C4：Splitter 集成（调用 Libs）

- 目标：在 Pipeline 中集成 `libs.splitter`，验证 Splitter 工厂配置是否生效。
- 修改文件：
  - `src/ingestion/pipeline.py`
  - `tests/unit/test_ingestion_splitter_integration.py`
- 验收标准：通过配置切换（例如改变 `chunk_size`），Ingestion Pipeline 产出的 chunk 长度发生相应变化。
- 测试方法：`pytest -q tests/unit/test_ingestion_splitter_integration.py`。

## C5：Transform 抽象基类 + ChunkRefiner（规则去噪 + 可选 LLM 重写）

- 目标：定义 `BaseTransform`；实现 `ChunkRefiner`：先做规则去噪，再支持（可选）LLM 重写/规范化，并提供可配置开关与失败降级（LLM 不可用/异常时不阻塞 ingestion）。
- 修改文件：
  - `src/ingestion/transform/base_transform.py`
  - `src/ingestion/transform/chunk_refiner.py`
  - `config/prompts/chunk_refinement.txt`（作为默认 prompt 来源；可在测试中注入替代文本）
  - `tests/unit/test_chunk_refiner.py`
- 验收标准：
  - 规则模式：能去掉空白/页眉页脚样式噪声（用 fixtures 字符串断言）。
  - LLM 模式：在注入/配置启用 LLM 时，会对 chunk 文本进行重写并返回稳定结果（测试中用 mock LLM 断言调用与输出）。
  - 降级行为：LLM 调用失败时回退到规则结果（可在 metadata 标记降级原因，但不抛出致命异常）。
- 测试方法：`pytest -q tests/unit/test_chunk_refiner.py`。

## C6：MetadataEnricher（规则增强 + 可选 LLM 增强 + 降级）

- 目标：实现元数据增强模块：提供规则增强的默认实现（例如从 chunk 文本抽取/推断 title、生成简短 summary、打 tags），并支持可选 LLM 增强（可配置开关 + 失败降级，不阻塞 ingestion）。
- 修改文件：
  - `src/ingestion/transform/metadata_enricher.py`
  - `tests/unit/test_metadata_enricher_contract.py`
- 验收标准：
  - 规则模式：输出 metadata 必须包含 `title/summary/tags`，且 `title/summary` 至少为非空字符串（fixtures 断言）。
  - LLM 模式：启用 LLM 时会调用一次增强逻辑，并将结果写回 metadata（测试中用 mock LLM 断言调用与输出）。
  - 降级行为：LLM 调用失败时回退到规则模式结果（可在 metadata 标记降级原因，但不抛出致命异常）。
- 测试方法：`pytest -q tests/unit/test_metadata_enricher_contract.py`。

## C7：ImageCaptioner（可选生成 caption + 降级不阻塞）

- 目标：实现 `image_captioner.py`：当启用 Vision LLM 且存在 image\_refs 时生成 caption 并写回 chunk metadata；当禁用/不可用/异常时走降级路径，不阻塞 ingestion。
- 修改文件：
  - `src/ingestion/transform/image_captioner.py`
  - `config/prompts/image_captioning.txt`（作为默认 prompt 来源；可在测试中注入替代文本）
  - `tests/unit/test_image_captioner_fallback.py`
- 验收标准：
  - 启用模式：存在 image\_refs 时会生成 caption 并写入 metadata（测试中用 mock Vision LLM 断言调用与输出）。
  - 降级模式：当配置禁用或异常时，chunk 保留 image\_refs，但不生成 caption 且标记 `has_unprocessed_images`。

- 测试方法：`pytest -q tests/unit/test_image_captioner_fallback.py`。

## C8：DenseEncoder ( 依赖 `libs.embedding` )

- 目标：实现 `dense_encoder.py`，把 `chunks.text` 批量送入 `BaseEmbedding`。
- 修改文件：
  - `src/ingestion/embedding/dense_encoder.py`
  - `tests/unit/test_dense_encoder.py`
- 验收标准：encoder 输出向量数量与 `chunks` 数量一致，维度一致。
- 测试方法：`pytest -q tests/unit/test_dense_encoder.py`。

## C9：SparseEncoder (BM25 统计与输出契约)

- 目标：实现 `sparse_encoder.py`：对 `chunks` 建立 BM25 所需统计 ( 可先仅输出 `term weights` 结构，索引落地下一步做 )。
- 修改文件：
  - `src/ingestion/embedding/sparse_encoder.py`
  - `tests/unit/test_sparse_encoder.py`
- 验收标准：输出结构可用于 `bm25_indexer`；对空文本有明确行为。
- 测试方法：`pytest -q tests/unit/test_sparse_encoder.py`。

## C10：BatchProcessor ( 批处理编排 )

- 目标：实现 `batch_processor.py`：将 `chunks` 分 batch，驱动 `dense/sparse` 编码，记录批次耗时 ( 为 `trace` 预留 )。
- 修改文件：
  - `src/ingestion/embedding/batch_processor.py`
  - `tests/unit/test_batch_processor.py`
- 验收标准：`batch_size=2` 时对 5 `chunks` 分成 3 批，且顺序稳定。
- 测试方法：`pytest -q tests/unit/test_batch_processor.py`。

## C11：VectorUpserter ( 幂等 upsert 契约 )

- 目标：实现 `vector_upserter.py`，生成稳定 `chunk_id` (`hash(source_path + section_path + content_hash)`)。
- 修改文件：
  - `src/ingestion/storage/vector_upserter.py`
  - `tests/unit/test_vector_upserter_idempotency.py`
- 验收标准：同一 `chunk` 两次 `upsert` 产生相同 `id`；内容变更 `id` 变更。
- 测试方法：`pytest -q tests/unit/test_vector_upserter_idempotency.py`。

## C12：BM25Indexer ( 倒排索引落地 )

- 目标：实现 `bm25_indexer.py`：把 `sparse encoder` 输出落盘到 `data/db/bm25/` ( 文件结构自行定义但需可重建 )。
- 修改文件：
  - `src/ingestion/storage/bm25_indexer.py`
  - `tests/unit/test_bm25_indexer_roundtrip.py`
- 验收标准：`build` 后能 `load` 并对同一语料查询返回稳定 `top ids`。

- 测试方法：`pytest -q tests/unit/test_bm25_indexer_roundtrip.py`。

### C13：ImageStorage ( 图片文件存储与索引表契约 )

- 目标：实现 `image_storage.py`：保存图片到 `data/images/{collection}/`，并记录 `image_id→path` 映射 ( 可先用 JSON/SQLite )。
- 修改文件：
  - `src/ingestion/storage/image_storage.py`
  - `tests/unit/test_image_storage.py`
- 验收标准：保存后文件存在；查找 `image_id` 返回正确路径。
- 测试方法：`pytest -q tests/unit/test_image_storage.py`。

### C14：Pipeline 编排 ( MVP 串起来 )

- 目标：实现 `pipeline.py`：串行执行 ( `integrity→load→split→transform→encode→store` )，并对失败步骤做清晰异常。
- 修改文件：
  - `src/ingestion/pipeline.py`
  - `tests/integration/test_ingestion_pipeline.py`
- 验收标准：对 fixtures 样例文档跑完整 pipeline，输出向量与 bm25 索引文件。
- 测试方法：`pytest -q tests/integration/test_ingestion_pipeline.py`。

### C15：脚本入口 ingest.py ( 离线可用 )

- 目标：实现 `scripts/ingest.py`，支持 `--collection`、`--path`、`--force`，并调用 pipeline。
- 修改文件：
  - `scripts/ingest.py`
  - `tests/e2e/test_data_ingestion.py`
- 验收标准：命令行可运行并在 `data/db` 产生产物；重复运行在未变更时跳过。
- 测试方法：`pytest -q tests/e2e/test_data_ingestion.py` ( 尽量用临时目录 )。

---

## 阶段 D：Retrieval MVP ( 目标：能 query 并返回 Top-K chunks )

### D1：QueryProcessor ( 关键词提取 + filters 结构 )

- 目标：实现 `query_processor.py`：关键词提取 ( 先规则/分词 )，并解析通用 filters 结构 ( 可空实现 )。
- 修改文件：
  - `src/core/query_engine/query_processor.py`
  - `tests/unit/test_query_processor.py`
- 验收标准：对输入 query 输出 `keywords` 非空 ( 可根据停用词策略 )，filters 为 dict。
- 测试方法：`pytest -q tests/unit/test_query_processor.py`。

### D2：DenseRetriever ( 调用 VectorStore.query )

- 目标：实现 `dense_retriever.py`，把 query embedding 与 filters 交给 VectorStore。
- 修改文件：
  - `src/core/query_engine/dense_retriever.py`

- `tests/unit/test_dense_retriever.py`
- 验收标准：当 `VectorStore` 返回候选列表时，`dense retriever` 透传并规范化 `score`。
- 测试方法：`pytest -q tests/unit/test_dense_retriever.py` (mock vector store)。

### D3：SparseRetriever (BM25 查询)

- 目标：实现 `sparse_retriever.py`：从 `data/db/bm25/` 载入索引并查询。
- 修改文件：
  - `src/core/query_engine/sparse_retriever.py`
  - `tests/unit/test_sparse_retriever.py`
- 验收标准：对已构建索引的 fixtures 语料，关键词检索命中预期 `chunk_id`。
- 测试方法：`pytest -q tests/unit/test_sparse_retriever.py`。

### D4：Fusion (RRF 实现)

- 目标：实现 `fusion.py`：RRF 融合 `dense/sparse` 排名并输出统一排序。
- 修改文件：
  - `src/core/query_engine/fusion.py`
  - `tests/unit/test_fusion_rrf.py`
- 验收标准：对构造的排名输入输出 `deterministic`；`k` 参数可配置。
- 测试方法：`pytest -q tests/unit/test_fusion_rrf.py`。

### D5：HybridSearch 编排

- 目标：实现 `hybrid_search.py`：并行/串行均可 (先串行)，调用 `dense+sparse+fusion`。
- 修改文件：
  - `src/core/query_engine/hybrid_search.py`
  - `tests/integration/test_hybrid_search.py`
- 验收标准：对 fixtures 数据，能返回 Top-K (包含 `chunk` 文本与 `metadata`)。
- 测试方法：`pytest -q tests/integration/test_hybrid_search.py`。

### D6：Reranker (Core 层编排 + fallback)

- 目标：实现 `core/query_engine/reranker.py`：接入 `libs.reranker` 后端，失败/超时回退 `fusion` 排名。
- 修改文件：
  - `src/core/query_engine/reranker.py`
  - `config/prompts/rerank.txt` (仅当启用 LLM Rerank 后端时使用)
  - `tests/unit/test_reranker_fallback.py`
- 验收标准：模拟后端异常时不影响最终返回，且标记 `fallback=true`。
- 测试方法：`pytest -q tests/unit/test_reranker_fallback.py`。

## 阶段 E：MCP Server 层与 Tools (目标：对外可用的 MCP tools)

### E1：MCP Server 入口与 Stdio 约束

- 目标：实现 `mcp_server/server.py`：遵循"stdout 只输出 MCP 消息，日志到 stderr"。
- 修改文件：

- `src/mcp_server/server.py`
- `tests/integration/test_mcp_server.py`
- 验收标准：启动 server 能完成 initialize；stderr 有日志但 stdout 不污染。
- 测试方法：`pytest -q tests/integration/test_mcp_server.py`（子进程方式）。

## E1.5：Protocol Handler 协议解析与能力协商

- 目标：实现 `mcp_server/protocol_handler.py`：封装 JSON-RPC 2.0 协议解析，处理 `initialize`、`tools/list`、`tools/call` 三类核心方法。
- 修改文件：
  - `src/mcp_server/protocol_handler.py`
  - `tests/unit/test_protocol_handler.py`
- 实现要点：
  - **ProtocolHandler 类：**
    - `handle_initialize(params)` → 返回 server capabilities（支持的 tools 列表、版本信息）
    - `handle_tools_list()` → 返回已注册的 tool schema（name, description, inputSchema）
    - `handle_tools_call(name, arguments)` → 路由到具体 tool 执行，捕获异常并转换为 JSON-RPC error
  - 错误码规范：遵循 JSON-RPC 2.0（-32600 Invalid Request, -32601 Method not found, -32602 Invalid params）
  - 能力协商：在 `initialize` 响应中声明 `capabilities.tools`
- 验收标准：
  - 发送 `initialize` 请求能返回正确的 `serverInfo` 和 `capabilities`
  - 发送 `tools/list` 能返回已注册 tools 的 schema
  - 发送 `tools/call` 能正确路由并返回结果或规范错误
- 测试方法：`pytest -q tests/unit/test_protocol_handler.py`。

## E2：实现 tool：query\_knowledge\_hub

### E2：实现 tool：query\_knowledge\_hub### E2：实现 tool：query\_knowledge\_hub

- 目标：实现 `tools/query_knowledge_hub.py`：调用 query engine，返回 Markdown + structured citations。
- 修改文件：
  - `src/mcp_server/tools/query_knowledge_hub.py`
  - `src/core/response/response_builder.py`
  - `src/core/response/citation_generator.py`
  - `tests/integration/test_mcp_server.py`（补用例）
- 验收标准：tool 返回 `content[0]` 为可读 Markdown；`structuredContent.citations` 含 `source/page/chunk_id/score`。
- 测试方法：`pytest -q tests/integration/test_mcp_server.py -k query_knowledge_hub`。

## E3：实现 tool：list\_collections

- 目标：实现 `tools/list_collections.py`：列出 `data/documents/` 下集合并附带统计（可延后到下一步）。
- 修改文件：
  - `src/mcp_server/tools/list_collections.py`

- `tests/unit/test_list_collections.py`
- 验收标准：对 fixtures 中的目录结构能返回集合名列表。
- 测试方法：`pytest -q tests/unit/test_list_collections.py`。

#### E4：实现 tool：get\_document\_summary

- 目标：实现 `tools/get_document_summary.py`：按 doc\_id 返回 title/summary/tags ( 可先从 metadata/缓存取 ) 。
- 修改文件：
  - `src/mcp_server/tools/get_document_summary.py`
  - `tests/unit/test_get_document_summary.py`
- 验收标准：对不存在 doc\_id 返回规范错误；存在时返回结构化信息。
- 测试方法：`pytest -q tests/unit/test_get_document_summary.py`。

#### E5：多模态返回组装 ( Text + Image )

- 目标：实现 `multimodal_assembler.py`：命中 chunk 含 image\_refs 时读取图片并 base64 返回 ImageContent。
- 修改文件：
  - `src/core/response/multimodal_assembler.py`
  - `tests/integration/test_mcp_server.py` ( 补图像返回用例 )
- 验收标准：返回 content 中包含 image type, mimeType 正确 · data 为 base64 字符串。
- 测试方法：`pytest -q tests/integration/test_mcp_server.py -k image`。

### 阶段 F：Observability + Evaluation ( 目标：可追踪 + 可回归 )

#### F1：TraceContext 数据结构与 record\_stage/finish

- 目标：实现请求级 trace：trace\_id、stages、metrics · 并能写入 jsonl。
- 修改文件：
  - `src/core/trace/trace_context.py`
  - `src/core/trace/trace_collector.py`
  - `tests/unit/test_trace_context.py`
- 验收标准：record\_stage 追加阶段；finish 输出 dict 可 JSON 序列化。
- 测试方法：`pytest -q tests/unit/test_trace_context.py`。

#### F2：结构化日志 logger (JSON Lines)

- 目标：实现 `observability/logger.py`：把 trace 写入 `logs/traces.jsonl`。
- 修改文件：
  - `src/observability/logger.py`
  - `tests/unit/test_jsonl_logger.py`
- 验收标准：写入一条 trace 后文件新增一行合法 JSON。
- 测试方法：`pytest -q tests/unit/test_jsonl_logger.py`。

#### F3：在关键路径打点 ( Query 与 Ingestion )



- 目标：在 Pipeline 与 HybridSearch/Rerank 中注入 TraceContext，利用 B 阶段抽象接口中预留的 `trace` 参数，显式调用 `trace.record_stage()` 记录各阶段数据。
- 修改文件：
  - `src/ingestion/pipeline.py`
  - `src/core/query_engine/hybrid_search.py`
  - `src/core/query_engine/reranker.py`
  - `tests/integration/test_hybrid_search.py` (断言 `trace` 中存在阶段)
- 说明：B 阶段的 `BaseEmbedding`、`BaseSplitter`、`BaseVectorStore`、`BaseReranker` 接口已预留 `trace: TraceContext | None = None` 参数，本任务负责在调用这些组件时传入实际的 `TraceContext` 实例。
- 验收标准：一次查询/一次摄取都会生成 `trace`，包含 `dense/sparse/fusion/rerank` 阶段耗时字段。
- 测试方法：`pytest -q tests/integration/test_hybrid_search.py`。

#### F4：Dashboard MVP (Streamlit)

- 目标：实现 `dashboard/app.py`：读取 `traces.jsonl`，展示请求列表与单条详情（最小可用）。
- 修改文件：
  - `src/observability/dashboard/app.py`
  - `scripts/start_dashboard.py`
- 验收标准：本地可启动并看到列表（手动验收）。
- 测试方法：手动运行 `python scripts/start_dashboard.py`（或 `streamlit run ...`）。

#### F5：Evaluation Runner + Golden Test Set 回归

- 目标：实现 `eval_runner.py`：读取 `tests/fixtures/golden_test_set.json`，跑 `retrieval` 并产出 `metrics`。
- 修改文件：
  - `src/observability/evaluation/eval_runner.py`
  - `scripts/evaluate.py`
  - `tests/integration/test_hybrid_search.py` (可增加黄金集 `smoke`)
- 验收标准：`evaluate` 脚本可运行，输出 `metrics`（至少 `custom` 指标）。
- 测试方法：`pytest -q tests/integration/test_hybrid_search.py` 或运行 `python scripts/evaluate.py`。

---

### 阶段 G：端到端验收与文档收口（目标：开箱即用的“可复现”工程）

#### G1：E2E：MCP Client 侧调用模拟

- 目标：实现 `tests/e2e/test_mcp_client.py`：以子进程启动 `server`，模拟 `tools/list + tools/call`。
- 修改文件：
  - `tests/e2e/test_mcp_client.py`
- 验收标准：完整走通 `query_knowledge_hub` 并返回 `citations`。
- 测试方法：`pytest -q tests/e2e/test_mcp_client.py`。

#### G2：E2E：Recall 回归（黄金集）

- 目标：实现 `tests/e2e/test_recall.py`：基于 `golden set` 做最小召回阈值（例如 `hit@k`）。
- 修改文件：



- `tests/e2e/test_recall.py`
- `tests/fixtures/golden_test_set.json` ( 补齐若干条 )
- 验收标准：hit@k 达到阈值 ( 阈值写死在测试里，便于回归 )。
- 测试方法：`pytest -q tests/e2e/test_recall.py`。

### G3：完善 README ( 运行说明 + 测试说明 + 常见问题 )

- 目标：让新用户能在 10 分钟内跑通 ingest + query + dashboard + tests。
- 修改文件：
  - `README.md`
- 验收标准：README 包含：安装、配置、摄取、查询、运行测试、启动 dashboard。
- 测试方法：按 README 手动走一遍 ( 并在 PR/自测中记录 )。

### G4：清理接口一致性 ( 契约测试补齐 )

- 目标：为关键抽象 ( VectorStore / Reranker / Evaluator ) 补齐契约测试，防止接口漂移。
  - 修改文件：
    - `tests/unit/test_vector_store_contract.py` ( 补齐边界 )
    - `tests/unit/test_reranker_factory.py` ( 补齐边界 )
    - `tests/unit/test_custom_evaluator.py` ( 补齐边界 )
  - 验收标准：`pytest -q` 全绿，且 contract tests 覆盖主要输入输出形状。
  - 测试方法：`pytest -q`。
- 

### 交付里程碑 ( 建议 )

- **M1 ( 完成阶段 A+B )**：工程可测 + 可插拔抽象层就绪，后续实现可并行推进。
- **M2 ( 完成阶段 C )**：离线摄取链路可用，能构建本地索引。
- **M3 ( 完成阶段 D+E )**：在线查询 + MCP tools 可用，可在 Copilot/Claude 中调用。
- **M4 ( 完成阶段 F+G )**：可观测 + 可回归 + 文档完善，形成“面试/教学/演示”可复现项目。

## 7. 可扩展性与未来展望

### 7.1 云端部署与后端架构学习

虽然当前阶段我们主要采用“本地运行”模式，但本项目的架构设计完全支持向云端迁移。这也是一个极佳的学习后端工程化的切入点。

- **Server 容器化**：计划编写 Dockerfile，将 MCP Server 打包为容器。这让我们有机会深入理解 Python 环境隔离、依赖管理以及 Docker 的最佳实践。
- **云端接入**：未来可以将 Server 部署至 Azure Container Apps 或 AWS Lambda。
  - **挑战与学习点**：处理网络延时、配置 API Gateway、增加 AuthN/AuthZ 鉴权机制 ( 保护私有数据不被公开访问 )。
- **多租户与并发**：从单用户本地服务转变为支持团队共享的服务。
  - **学习点**：在 Chroma 中实现 Namespace 隔离、处理并发请求锁、优化 embedding 缓存策略。

### 7.2 业务深耕：从“通用”到“垂直” (Vertical Domain Adaptation)

RAG 系统的上限取决于其对特定业务数据的理解深度。未来的核心扩展方向是将通用的技术框架与具体的业务场景深度结合。在将本项目应用到实际生产环境时，识别并解决以下“最后一公里”的难题，将是提升系统价值

的关键：

- **多源异构数据的复杂适配：**
  - 现实业务中不仅有 PDF，还大量存在 PPTX, DOCX, XLSX 甚至 HTML 数据。
  - **挑战：**如何处理不同格式的特有语义？例如 PPT 中的演讲者备注往往比正文更关键，Excel 中的公式逻辑与跨行关联如何保留？目前的通用处理方式容易丢失这些“隐性知识”，未来需要针对每种格式探索更深度的解析能力。
- **复杂结构化数据的精确理解：**
  - 简单的文本切分（Chunking）在处理表格、层级列表时往往会破坏语义。
  - **挑战：**
    - **表格理解：**如何处理跨页长表格、合并单元格以及含有复杂表头的财务报表？如果切分不当，检索时只能找到数字却不知道对应的列名（指标含义）。
    - **上下文断裂：**当一个完整的逻辑段落（如合同条款）被切分到两个 chunk 时，如何保证检索其中一段时能感知到整体的上下文约束？
- **业务逻辑驱动的生成控制：**
  - 仅仅根据“相似度”召回文档在企业级场景中往往不够。
  - **挑战：**
    - **时效性与版本管理：**当知识库中同时存在“2023版”和“2024版”规章时，如何确保系统不会混淆历史数据与最新标准？
    - **权限与受众适配：**面对内部员工与外部客户，如何控制生成答案的详略程度与敏感信息披露？
    - **拒答机制：**当召回内容的置信度不足时，如何让系统诚实地回答“不知道”而不是基于相关性较低的片段强行拼凑答案（幻觉问题）？

### 7.3 迈向自主智能：Agentic RAG 的演进路径

当前的 RAG 架构主要遵循“一次检索-一次生成”的固有范式，但在面对极其复杂的问题（如跨文档对比、多步推理）时，单一的线性流程往往力不从心。本项目作为标准的 MCP Server，天然具备向 **Agentic RAG（代理式 RAG）** 演进的潜力。这不需要重写现有代码，而是通过在 Server 端提供更细粒度的工具，赋能 Client 端的 Agent 具备更强的自主性：

- **从“单步检索”到“多步决策”：**
  - 目前 Agent 可能只调用一个通用的 **search** 工具。
  - **未来演进：**Server 可以暴露如 **list\_directory**（查看目录结构）、**preview\_document**（预览摘要）、**verify\_fact**（事实核查）等更原子化的工具。Agent 可以像人类研究员一样，先看目录圈定范围，再针对性阅读，最后交叉验证信息，从而解决复杂问题。
- **让 Agent 具备“反思”能力：**
  - **未来演进：**利用现有的评估模块，Server 可以提供一个 **self\_check** 接口。Agent 在生成答案后，可以自主调用该接口检测是否存在幻觉，或者检索结果是否真正支撑了论点。如果发现不足，Agent 可以自主决定进行第二轮更深度的搜索。
- **动态策略选择：**
  - **未来演进：**不再硬编码使用混合检索。Server 可以将 **keyword\_search** 和 **semantic\_search** 作为独立工具暴露。Agent 可以根据用户意图自主判断：如果是搜人名，只用关键词搜；如果是搜概念，通过语义搜。这种工具使用的灵活性正是 Agentic RAG 的核心魅力。

这种演进方向将把本项目从一个“智能搜索引擎”升级为一个“智能研究助理”的基础设施底座。